

A Decade of Code Comment Quality Assessment: A Systematic Literature Review ¹

Pooja Rani^a, Arianna Blasi^b, Nataliia Stulova^a, Sebastiano Panichella^c, Alessandra Gorla^d, Oscar Nierstrasz^a ²

^aSoftware Composition Group, University of Bern, Bern, Switzerland ³

^bUniversità della Svizzera italiana, Lugano, Switzerland

^cZurich University of Applied Science, Zurich, Switzerland

^dIMDEA Software Institute, Madrid, Spain

Abstract ⁴

Code comments are important artifacts in software systems and play a paramount role in many software engineering (SE) tasks ⁵ related to maintenance and program comprehension. However, while it is widely accepted that high quality matters in code comments just as it matters in source code, *assessing* comment quality in practice is still an open problem. First and foremost, there is no unique definition of quality when it comes to evaluating code comments. The few existing studies on this topic rather focus on specific attributes of quality that can be easily quantified and measured. Existing techniques and corresponding tools may also focus on comments bound to a specific programming language, and may only deal with comments with specific scopes and clear goals (*e.g.*, Javadoc comments at the method level, or in-body comments describing TODOs to be addressed).

In this paper, we present a Systematic Literature Review (SLR) of the last decade of research in SE to answer the following ⁶ research questions: (i) What *types of comments* do researchers focus on when assessing comment quality? (ii) What *quality attributes* (QAs) do they consider? (iii) Which *tools and techniques* do they use to assess comment quality?, and (iv) How do they *evaluate* their studies on comment quality assessment in general?

Our evaluation, based on the analysis of ²³⁵³ papers and the actual review of ⁴⁷ relevant ones, shows that (i) most studies and techniques focus on comments in Java code, thus may not be generalizable to other languages, and (ii) the analyzed studies focus on four main QAs of a total of 21 QAs identified in the literature, with a clear predominance of checking *consistency* between comments and the code. We observe that researchers rely on manual assessment and specific heuristics rather than the automated assessment of the comment quality attributes, with evaluations often involving surveys of students and the authors of the original studies but rarely professional developers. ⁷

Keywords: code comments, documentation quality, systematic literature review ⁸

1. Introduction ⁹

Software systems are often written in several programming ¹⁰ languages [1], and interact with many hardware devices and software components [2, 3]. To deal with such complexity and to ease maintenance tasks, developers tend to document their

software with various artifacts, such as design documents and ¹¹ code comments [4]. Several studies have demonstrated that *high quality* code comments can support developers in software comprehension, bug detection, and program maintenance activities [5, 6, 7]. However, code comments are typically written using natural language sentences, and their syntax is neither imposed by a programming language's grammar nor checked by its compiler. Additionally, static analysis tools and linters provide limited syntactic support to check comment quality. There-

Email addresses: pooja.rani@unibe.ch (Pooja Rani),
arianna.blasi@usi.ch (Arianna Blasi),
nataliia.stulova@inf.unibe.ch (Nataliia Stulova), panc@zhaw.ch
(Sebastiano Panichella), alessandra.gorla@imdea.org (Alessandra
Gorla), oscar.nierstrasz@unibe.ch (Oscar Nierstrasz)

fore, writing high-quality comments and maintaining them in projects is a responsibility mostly left to developers [8, 9].

The problem of *assessing the quality of code comments* has gained a lot of attention from researchers during the last decade [10, 11, 12, 13, 14]. Despite the research community's interest in this topic, there is no clear agreement on what quality means when referring to code comments. Having a general definition of quality when referring to code comments is hard, as comments are diverse in purpose and scope.

Problem Statement. Maintaining high-quality code comments is vital for software evolution activities, however, *assessing the overall quality of comments is not a trivial problem*. As developers use various programming languages, adopt project-specific conventions to write comments, embed different kinds of information in a semi-structured or unstructured form [15, 13], and lack quality assessment tools for comments, ensuring comment quality in practice is a complex task. Even though specific comments follow all language-specific guidelines in terms of syntax, it is still challenging to determine automatically whether they satisfy other quality aspects, such as whether they are consistent or complete with respect to the code or not [16]. There are various such aspects, *e.g.*, readability, content relevance, and correctness that should be considered when assessing comments, but tools do not support all of them. Therefore, a comprehensive study of the specific attributes that influence code comment quality and techniques proposed to assess them is essential for further improving comment quality tools.

Previous mapping and literature review studies have collected numerous quality attributes (QAs) that are used to assess the quality of software documentation based on their importance and effect on the documentation quality. Ding *et al.* [17] focused specifically on software architecture and requirement documents, while Zhi *et al.* [18] analyzed code comments along with other types of documentation, such as requirement and design documents. They identified 16 QAs that influence the quality of software documentation. However, the identified QAs are extracted from a body of literature concerning relatively old

studies (*i.e.*, studies conducted prior to the year 2011) and are limited in the context of code comments. For instance, only 10% of the studies considered by Zhi *et al.* concern code comments. Given the increasing attention that researchers pay to comment quality assessment, it is essential to know which QAs, tools and techniques they propose to assess code comment quality.

To achieve this objective, we perform an SLR on studies published in the last decade, *i.e.*, 2011-2020. We review 2353 studies and find 47 to be relevant to assessing comment quality. From these we extract the programming language, the types of analyzed comments, QAs for comments, techniques to measure them, and the preferred evaluation type to validate their results.

We observe that (i) most of the studies and techniques focus on comments in Java code, (ii) many techniques that are used to assess QAs are based on heuristics and thus may not be generalizable to other languages, (iii) a total of 21 QAs are used across studies, with a clear dominance of *consistency*, *completeness*, *accuracy*, and *readability*, and (iv) several QAs are often assessed manually rather than with the automated approaches. We find that the studies are typically evaluated by measuring performance metrics and surveying students rather than by performing validations with practitioners. This shows that there is much room for improvement in the state of the art of comment quality assessment.

The **contributions** of this paper are:

- i) an SLR of a total of 2353 papers, of which we review the 47 most relevant ones, focusing on QAs mentioned and research solutions proposed to assess code comment quality,
- ii) a catalog of 21 QAs of which four QAs are often investigated, while the majority is rarely considered in the studies, and of which 10 are new with respect to the previous study by Zhi *et al.* [18],
- iii) a catalog of methods used to measure these 21 QAs in research studies,
- iv) an overview of the approaches and tools proposed to assess

comment quality, taking into account the types of comments and the programming languages they consider,

v) a discussion of the challenges and limitations of approaches and tools proposed to assess different and complementary comment QAs, and

vi) a publicly available dataset including all validated data, and steps to reproduce the study in the replication package.¹

Paper structure. The rest of the paper is organized as follows. In section 2 we highlight our motivation and rationale behind each research question, and we present our methodology, including the different steps performed to answer our research questions. In section 3 we report the study results. We discuss the results in section 4 and their implications and future direction in section 5. We highlight the possible threats to validity for our study in section 6. Then section 7 summarizes the related work, in relation to the formulated research questions. Finally, section 8 concludes our study, outlining future directions.

2. Study Design

The main objective of our study is to present an overview of the state of the art in assessing the quality of code comments. Specifically, we aim to highlight the QAs mentioned in the literature, and the techniques used so far to assess comment quality. To this end, we carry out an SLR, following the widely accepted guidelines of Kitchenham *et al.* [19] and Keele [20]. The first step in this direction is to specify the research questions related to the topic of interest [19]. The following steps focus on finding a set of relevant studies that are related to the research questions based on an unbiased search strategy.

2.1. Research questions

Our goal is to foster research that aims at building code comment assessment tools. To achieve this goal, we conduct an SLR, investigating the literature of the last decade to identify

comment related QAs and solutions that address related challenges. We formulate the following research questions:

- **RQ₁:** *What types of comments do researchers focus on when assessing comment quality?* **Motivation:** Comments are typically placed at the beginning of a file, usually to report licensing or author information, or placed preceding a class or function to document the overview of a class or function and its implementation details. Depending on the specific type of comment used in source code and the specific programming language, researchers may use different techniques to assess them. These techniques may not be generalizable to other languages. For example, studies analyzing class comments in object-oriented programming languages may need extra effort to generalize the comment assessment approach to functional programming languages. We, therefore, investigate the comment types researchers target.

- **RQ₂:** *What QAs do researchers consider in assessing comment quality?*

Motivation: QAs may solely concern syntactic aspects of the comments (*e.g.*, syntax of comments), writing style (*e.g.*, grammar), or content aspects (*e.g.*, consistency with the code). Researchers may use different terminology for the same QA and thus these terms must be mapped across studies to obtain a unifying view of them, for instance, if the *accuracy* QA is defined consistently across studies or another terminology is used for it. We collect all the possible QAs that researchers refer to and map them, if necessary, following the methodology of Zhi *et al.*. Future studies that aim to improve specific aspects of comment quality evaluation can use this information to design their tools and techniques.

- **RQ₃:** *Which tools and techniques do researchers use to assess comment QAs?*

Motivation: Researchers may assess QAs manually, or may use sophisticated tools and techniques based on simple heuristics or complex machine learning (ML) to as-

¹<https://doi.org/10.5281/zenodo.4729054>

sess them automatically. We aim to identify if there are clear winning techniques for this domain and collect various metrics and tools used for this purpose.

• **RQ₄:** *What kinds of contribution do studies often make?*

Motivation: Engineering researchers usually motivate their research based on the utility of their results. Auyang clarifies that engineering aims to apply scientific methods to real world problems [21]. However, software engineering currently lacks validation [22]. With this question, we want to understand what types of solution researchers contribute to improving automatic comment quality assessment, such as metrics, methods, or tools. This RQ can provide insight into specific kinds of solutions for future work.

• **RQ₅:** *How do researchers evaluate their comment quality assessment studies?*

Motivation: Researchers may evaluate their comment assessment approaches, *e.g.*, by surveying developers, or by using a dataset of case studies. However, how often they involve professional developers and industries in such studies is unknown.

2.2. Search Strategy

After formulating the research questions, the next steps focus on finding relevant studies that are related to the research questions. In these steps, we

1. construct search keywords in subsection 2.2.1,
2. choose the search timeline in subsection 2.2.2,
3. collect sources of information in subsection 2.2.3,
4. retrieve studies in subsection 2.2.4,
5. select studies based on the inclusion/exclusion criteria in subsection 2.2.5, and
6. evaluate the relevant studies to answer the research questions in subsection 2.2.6.

2.2.1. Search Keywords

Kitchenham *et al.* recommended formulating individual facets or search units based on the research questions [19]. These search units include abbreviations, synonyms and other spellings, and they are combined using boolean operators. Pettricrew *et al.* suggested PIO (population, interventions, and outcome) criterion to define such search units [23].

The *populations* include terms related to the standards. We first examine the definitions of *documentation* and *comment* in *IEEE Standard Glossary of Software Engineering Terminology* (IEEE Standard 610.12-1990) to collect the main keywords. According to the definition, we identify the keywords *comment*, *documentation*, and *specification* and add them to the set K_1 . We further add frequently mentioned comment-related keywords, such as *API*, *annotation*, and *summar* to the set K_1 .

The *interventions* include terms that are related to software methodology, tools, technology, or procedures. With respect to quality assessment, we define the intervention keywords to be *quality*, *assess*, *metric*, *measure*, *score*, *analy*, *practice*, *structure*, *study*, or *studied* and add them to the set K_2 .

Note that we add common variations of the words manually, for example, we add “summar” keyword to the set to cover both “summary” and “summarization”. We do not use any NLP libraries to stem words due to two main reasons, (i) to reduce the noisy matches, and (ii) the words from the title and abstract of the papers are not preprocessed (stemmed or lemmatized), therefore stemming the keywords might not find the exact or prefix matches. For example, using the porter stemming approach, the word “study” will be stemmed to “studi” and we might miss the papers with “study” word. To avoid such cases, we add common variations of this word *study* and *studied* to our search keywords.

The *outcomes* include terms that are related to factors of significance to developers (*e.g.*, reduced cost, reduced time to assess quality). Since it is not a required unit to restrict the search scope, and our focus is on all kinds of quality assessment approaches, we exclude the outcomes in our search keywords. However, to narrow down our search and exclude irrelevant pa-

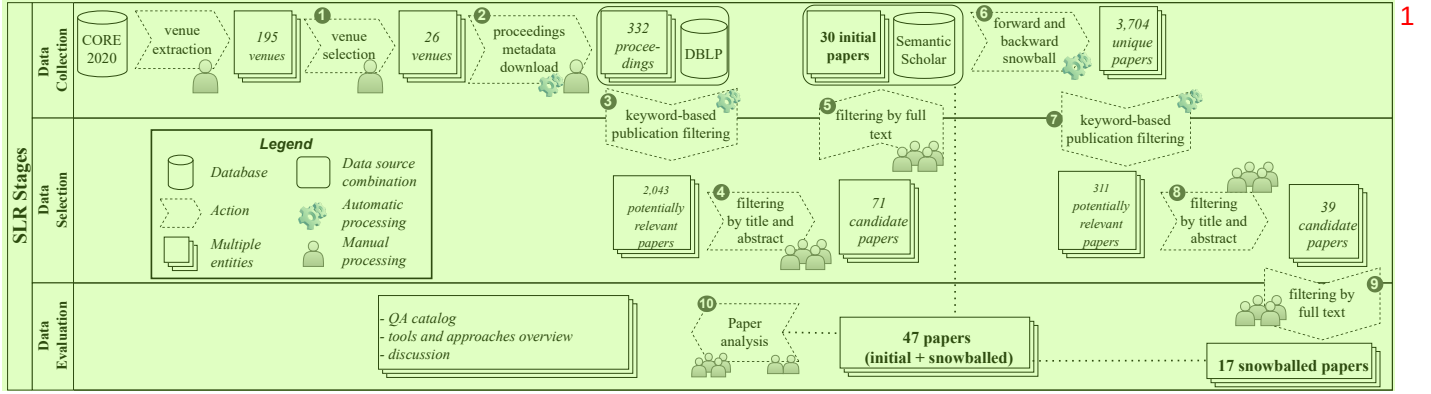


Figure 1: SLR stages to collect relevant papers

Table 1: keywords selected according to PIO criterion

Criteria	keywords
Populations (K_1)	<i>comment, documentation, specification, API, annotation, and summary</i>
Interventions (K_2)	<i>quality, assess, metric, measure, score, analy, practice, structur, study, and studied</i>

pers, such those about code reviews or testing, or non-technical papers, we formulate another set of keywords, K_3 . In this set, we include *code review, test, keynote, invited, and poster*, to exclude entries of non-technical papers that were not filtered out using the heuristics on the number of pages.

Hence, using the final set of keywords (also given in Table 1), we select a paper if its title and abstract match the keywords from K_1 and K_2 but not from K_3 where the prefix function is used to match the keywords in the paper.

2.2.2. Timeline

We focus our SLR on the last decade (*i.e.*, January 2011-December 2020) since Zhi *et al.* investigated the works on software documentation quality — including code comments — from 1971 to 2011 [18]. Our results can thus be used to observe the evolution of comment quality assessment, but, more importantly, they naturally complement the existing body of knowledge on the topic.

We then proceed to the main steps *i.e.*, retrieving the paper

data, selecting venues, and identifying the relevant papers for our comment context.

2.2.3. Data collection

Concretely, our data collection approach comprises three main steps, *i.e.*, literature data collection, data selection, and data evaluation, which we sketch in Figure 1 and present in further detail as follows:

We now describe how we automatically collect the data from the literature, explaining the rationale behind our selection of venues and our automatic keyword-based filtering to identify the likely relevant papers regarding comment quality assessment. We justify the need for another step of data gathering based on the snowball approach in Section 2.2.5. Finally, we present our criteria for the careful evaluation of the relevant papers in Sec 2.2.6.

Venue Selection. Code comment analysis, generation, usage, and maintenance are of primary interest to the SE research community. Thus, in order to systematically review the literature on the comment quality assessment, we start by focusing on the SE venues. We use the latest 2020 updated version of the conference and journal database of the CORE ranking portal as a primary data source to identify all the potentially relevant SE venues.² The portal provides assessments of major conferences and journals in the computing disciplines, and it is a well-established and regularly-validated registry maintained

²<https://www.core.edu.au/conference-portal>

by the academic community. We extract all ranked journals in SE (search code 803) from the CORE portal³ and all top conferences and workshops in the SE field (search code 4612).⁴ This process gives us an initial list of 85 journal and 110 conference venues. We select in step 1 26 software engineering (SE) conferences and journals from 195 candidate venues based on the likelihood of finding relevant papers in their proceedings.

We focus on A* and A conferences and journals, and add conferences of rank B or C if they are co-located with previously selected A* and A conferences to have venues, such as the *IEEE/ACM International Conference on Program Comprehension* (ICPC) or the *IEEE International Workshop on Source Code Analysis and Manipulation* (SCAM) that focus on source code comprehension and manipulation.

We prune venues that may not contain relevant contributions to source code comments. Specifically, we exclude a venue if its ten years of proceedings contain fewer than five occurrences of the words *documentation* or *comment*. This way, we exclude conferences, such as *IEEE International Conference on Engineering of Complex Computer Systems* (ICECCS), *Foundations of Software Science and Computational Structures* (FoSSaCS), and many others that primarily focus on other topics, such as verification or programming languages. Thus, we reduce our dataset to 20 conferences and six journals, as shown in Table 2.

In Table 2, the column *Type* specifies whether a venue is a conference (C) or a journal (J), and the column *Rank* denotes the corresponding CORE rank of the venue as of April 2021. The column *Selection* indicates the data collection phase in which the venue was first selected. The column *Papers per venue* indicates the total number of papers selected from this venue, both during the direct search and the snowball search.

We consider only full papers (published in a technical track and longer than five pages) since they are likely to be an extended or mature version of the papers published in other tracks,

³<http://portal.core.edu.au/jnl-ranks/?search=803&by=for&source=CORE2020&sort=arank&page=1> accessed on 25 Mar, 2021

⁴<http://portal.core.edu.au/conf-ranks/?search=4612&by=for&source=CORE2020&sort=arank&page=1> accessed on 25 Mar, 2021

such as NIER, ERA, or Poster.

2.2.4. Data Retrieval

We retrieve in step 2 the proceedings from January 2011 to December 2020 of the selected venues from the DBLP digital library. From each paper, we collect its metadata using the GitHub repository⁵, such as the title, authors, conference track (if present), its page length, and its Digital Object Identifier (DOI), directly from DBLP for a total of 17554 publications. For each paper, the DOI is resolved and its abstract is collected from the publisher webpage.

Keyword-based filtering. We apply in step 3 a keyword-based search (given in subsection 2.2.1) using a prefix function to the retrieved proceedings to select potentially relevant papers. We account for possible upper- and lowercase letters in the keywords, and sometimes use variations of keywords (*e.g.*, singular and plural forms).

Our filtering will get papers (whose title and abstract include keywords from K_1 and K_2 but not from K_3) that explicitly mention concepts we are interested in, *e.g.*, “A Human Study of Comprehension and Code Summarization” from ICPC 2020 [24] is matched by keywords *summar* from K_1 in the title and *quality* from K_2 in the abstract, but will exclude papers not sufficiently close to our research subject, *e.g.*, “aComment: mining annotations from comments and code to detect interrupt related concurrency bugs” from ICSE 2011 has two keywords *comment* and *annotation* from K_1 but none from the K_2 .

The final set of keywords we use for filtering is the result of an iterative approach: we manually scan the full venue proceedings metadata to make sure the set of keywords did not prune relevant papers, and we refine the set of keywords during several iterative discussions. This iterative approach gives us confidence that our keyword-based filtering approach does not lead to false negatives for the selected venues. After applying the keyword-based filtering, we identify 2043 studies as potentially-relevant papers from a total of 17554, which we review manually.

⁵<https://github.com/sbaltes/dblp-retriever>

Table 2: Included Journals, Conferences, and Workshops. 1

Venue	Abbreviation	Rank	Type	Selection	Papers per venue
ACM Computing Surveys	CSUR	A*	J	search	
ACM Transactions on Software Engineering and Methodology	TOSEM	A*	J	search	
IEEE Transactions on Software Engineering	TSE	A*	J	search	5
Empirical Software Engineering: an international journal	EMSE	A	J	search	6
Journal of Systems and Software	JSS	A	J	search	2
Information and Software Technology	IST	A	J	search	1
ACM SIGSOFT Symposium on the Foundations of Software Engineering	ESEC/FSE	A*	C	search	3
International Conference on Software Engineering	ICSE	A*	C	search	6
Architectural Support for Programming Languages and Operating Systems	ASPLOS	A*	C	search	
Computer Aided Verification	CAV	A*	C	search	
International Conference on Functional Programming	ICFP	A*	C	search	
ACM Conference on Object Oriented Programming Systems Languages and Applications	OOPSLA	A*	C	search	1
ACM-SIGPLAN Conference on Programming Language Design and Implementation	PLDI	A*	C	search	
ACM-SIGACT Symposium on Principles of Programming Languages	POPL	A*	C	search	
Measurement and Modeling of Computer Systems	SIGMETRICS	A*	C	search	
Automated Software Engineering Conference	ASE	A	C	search	2
International Conference on Evaluation and Assessment in Software Engineering	EASE	A	C	search	1
International Symposium on Empirical Software Engineering and Measurement	ESEM	A	C	search	1
IEEE International Conference on Software Maintenance and Evolution	ICSME	A	C	search	2
IEEE International Working Conference on Mining Software Repositories	MSR	A	C	search	1
International Symposium on Software Reliability Engineering	ISSRE	A	C	search	
IEEE International Working Conference on Software Visualisation	VISSOFT	B	C	search	
IEEE International Conference on Global Software Engineering	ICGSE	C	C	search	
IEEE International Conference on Program Comprehension	ICPC	C	C	search	5
International Workshop on Modelling in Software Engineering	MISE	C	C	search	
IEEE International Workshop on Source Code Analysis and Manipulation	SCAM	C	C	search	1
International Conference on Web Information Systems and Applications	WISA	-	C	snowball	1
Software Quality Journal	-	C	J	snowball	1
Software: Practice and Experience	SPE	B	J	snowball	1
ACM Symposium on Applied Computing	SAC	B	C	snowball	1
IEEE Workshop on Machine Learning Techniques for Software Quality Evaluation	MaLTeSQuE	-	C	snowball	1
Journal of Software: Evolution and Process	JSEP	B	J	snowball	1
Asia-Pacific Symposium on Internetware	Internetware	-	C	snowball	1
IEEE International Joint Conference on Neural Networks	IJCNN	A	C	snowball	1
International Computer Software and Applications Conference	COMPSAC	B	C	snowball	1
Asia-Pacific Software Engineering Conference	APSEC	B	C	snowball	1
International journal of software engineering and knowledge engineering	SEKE	-	J	snowball	1

2.2.5. Data selection 1

We analyze 4 the 2043 selected papers following the protocol where four authors or evaluators manually evaluate the papers based on the inclusion and exclusion criterion to ensure that they indeed assess comment quality. 2

Inclusion criteria 3

- I1 The topic of the paper is about code comment quality. 4
- I2 The study presents a model/technique/approach to assess code comments or software documentation including code comments.

Exclusion criteria 5

- E1 The paper is not in English. 6
- E2 It does not assess any form of quality aspects of comments *e.g.*, content, style, or language used.
- E3 It is not published in a technical track.
- E4 It is a survey paper.
- E5 It is not a peer reviewed paper, or it is a pre-print.
- E6 It covers other documentation artifacts, *i.e.*, not comments.
- E7 It is shorter than 5 pages.

Manual analysis. The selected papers were equally divided among four evaluators (*i.e.*, two Ph.D. candidates and two faculty members) based on years of publications so that each evaluator gets papers from all venues, *e.g.*, the first author evaluate proceedings from 2011 to 2013. We make sure that evaluators do not take decisions on papers they co-authored to avoid conflicts of interest. Each evaluator has at least two years of experience in the domain of comment analysis. Each paper is reviewed by three evaluators. The evaluators follow a three-iteration-based process to evaluate the assigned papers. In the first iteration, the first evaluator independently assesses the relevance of a paper based on the criteria by inspecting each paper’s title and abstract, to make an initial guess, then inspecting its

conclusion to reach the final decision. In the next iteration, another evaluator reviews the paper and validates the previous decision by adding the label “agrees/disagrees with the first evaluator”. With this process, every publication selected in the final set is reviewed by at least two researchers. In case they do not agree, the third evaluator reviews it [25], and the final decision is taken based on the majority voting mechanism. 8

We decide, for instance, to include the study by Hata *et al.* [26], even though it only talks about links in comments. Though it does not explicitly describe any quality aspect of comments, it mentions the traceability of the links, which is a QA we consider in our study. All studies considered in our SLR together with their evaluation (the agreement and disagreement for each study) are available in our replication package. 9

Thus, we reduce 2043 papers to 71 candidate papers (*i.e.*, 3%) with a fair agreement according to Cohen’s Kappa ($k=0.36$). For all candidate papers, we read in step 5 their introduction, conclusion, and the study design (if needed), and discuss them amongst ourselves to ensure their relevance. During this analysis process, some additional papers were found to be irrelevant. For example, the study by Aghajani *et al.* seems relevant based on the title and abstract, but does not really evaluate code comments, and we thus discarded it [27]. With this process, 41 papers in total were discarded, reducing the relevant paper set to 30 papers. 10

Data gathering for snowballing. To include further relevant papers that we might have missed with the venue-based approach, we perform in step 6 a forward and backward snowballing approach for the 30 papers and retrieve a total of 3704 unique papers. 11

Snowball papers	Total	Unique	Selected
from citations	2610	1624	741
from references	3369	2080	2021

 12

The column *Total* reports the total number of references and citations collected. The *Unique* column reports a total number of unique items (*i.e.*, since relevant papers cover similar topics many references, and citations are shared across our set of 13

studies). Finally, the column *Selected* reports the total number of unique references and citations whose publication year falls within our time frame range, *i.e.*, 2011-2020.

Data selection from snowballing. We repeat in step 7 the same keyword-based filtering to these 3704 papers, as described in subsection 2.2.3. As a result, 311 papers were added for manual analysis. We repeat in step 8 the three-iteration based manual analysis process and find 39 additional candidate papers to analyze. After the second round of discussion 9 we keep 17 additional relevant papers. We find a total of 47 papers shown in Table 3 published in the venues shown in Table 2. In Table 3, the column *Study ID* indicates the ID assigned to each paper, the column *Title* presents the title of the paper, and the column *Year* indicates the years in which the paper is published.

To further ensure the relevance of our search strategy, we search our keywords on popular publication databases, such as ACM, IEEE Xplore, Wiley *etc.* We search for our keywords in titles and abstracts.⁶ We retrieve 13 144 results from IEEE Xplore, and 10 567 from ACM for the same timeline (2011-2020). We inspect first 200 results (sorted by relevance criterion on the publisher webpage) from each of these databases. We apply our inclusion and exclusion criterion to find the extent to which our venue selection criteria might have missed relevant papers. Our results from ACM show that 19% of the these papers are already covered by our search strategy but only 5% of them fulfilled our inclusion criterion. Nearly 81% of the papers are excluded due to their non-SE venue. Among these papers, 80% are unrelated to the code comment quality aspect while 1% of papers (two papers) that are related to code comments are missed due to two main reasons, (i) the venue not being indexed in CORE2020, and (ii) the paper being from a non-technical track. Similarly, the results from IEEE show that 30% of the papers are already covered by our search strategy but only 5% of them fulfilled the inclusion criterion. Nearly 69% of the papers are excluded due to their non-SE venue and unrelated to code comment quality aspect. We also find 1% pa-

pers that are relevant to our topic of interest but excluded due to the length criteria, specifically one of the paper is a poster paper and another is a short paper.

2.2.6. Data Evaluation 5

We work in step 10 on the full versions of the 47 relevant papers to identify the QAs and the approaches to assess comments. In case we cannot retrieve the full PDF version of a paper, we use university resources to access it. This affects only one paper by Sun *et al.*, which requires payment to access the full version [41]. In case we cannot access a paper via any resource, we remove it from our list. We find no such inaccessible study.

We report all papers in an online shared spreadsheet on Google Drive to facilitate their analysis collaboratively. For each paper we extract common metadata, namely *Publication year*, *Venue*, *Title*, *Authors*, *Authors' country*, and *Authors' affiliation*. We then extract various dimensions (described in the following paragraphs) formulated to answer all research questions.

2.3. Data extraction for research questions 8

To answer RQ1 (*What types of comments do researchers focus on when assessing comment quality?*), we record the *Comment scope* dimension. It lists the scope of comments under assessment such as class, API, method (function), package, license, or inline comments. In case the comment type is not mentioned, we classify it as “code comments”. Additionally, we identify the programming languages whose comments are analyzed, and record this in the *Language analyzed* dimension.

To answer RQ2 (*What QAs do researchers consider in assessing comment quality?*), we identify various QAs researchers mention to assess comment quality. This reflects the various quality aspects researchers perceive as important to have high-quality comments. Table 4 lists the QAs in the *Quality attribute (QA)* column and their brief summary in the *Description* column. Of these QAs, several are mentioned by

⁶It is not possible to search the keywords in abstracts in Wiley.

Table 3: Included studies ¹

Study ID	Title	Year	Reference ²
S01	How Good is Your Comment? A Study of Comments in Java Programs.	2011	[28]
S02	Quality Analysis of Source Code comments.	2013	[11]
S03	Evaluating Usage and Quality of Technical Software Documentation: An Empirical Study.	2013	[29]
S04	Inferring Method Specifications from Natural Language API Descriptions.	2012	[30]
S05	Using Traceability Links to Recommend Adaptive Changes for Documentation Evolution.	2014	[31]
S06	On Using Machine Learning to Identify Knowledge in API Reference Documentation.	2019	[32]
S07	Detecting Fragile Comments.	2017	[12]
S08	Automatically Assessing Code Understandability: How Far are We?	2017	[33]
S09	Analyzing APIs Documentation and Code to Detect Directive Defects.	2017	[16]
S10	The Effect of Poor Source Code Lexicon and Readability on Developers' Cognitive Load.	2018	[34]
S11	A Large-Scale Empirical Study on Linguistic Antipatterns Affecting APIs.	2018	[35]
S12	Improving API Caveats Accessibility by Mining API Caveats Knowledge Graph.	2018	[36]
S13	A Learning-Based Approach for Automatic Construction of Domain Glossary from Source Code and Documentation.	2019	[37]
S14	A Framework for Writing Trigger-Action Todo Comments in Executable Format.	2019	[38]
S15	A Large-Scale Empirical Study on Code-Comment Inconsistencies.	2019	[14]
S16	Software Documentation Issues Unveiled.	2019	[39]
S17	The Secret Life of Commented-Out Source Code.	2020	[40]
S18	Code Comment Quality Analysis and Improvement Recommendation: An Automated Approach	2016	[41]
S19	A Human Study of Comprehension and Code Summarization.	2020	[24]
S20	CPC: Automatically Classifying and Propagating Natural Language Comments via Program Analysis.	2020	[42]
S21	Recommending Insightful Comments for Source Code using Crowdsourced Knowledge.	2015	[43]
S22	Improving Code Readability Models with Textual Features.	2016	[44]
S23	Automatic Source Code Summarization of Context for Java Methods.	2016	[45]
S24	Automatic Detection and Repair Recommendation of Directive Defects in Java API Documentation.	2020	[46]
S25	Measuring Program Comprehension: A Large-Scale Field Study with Professionals.	2018	[47]
S26	Usage and Usefulness of Technical Software Documentation: An Industrial Case Study	2015	[48]
S27	What Should Developers be Aware of? An Empirical Study on the Directives of API Documentation.	2012	[49]
S28	Analysis of License Inconsistency in Large Collections of Open Source Projects.	2017	[50]
S29	Classifying Code Comments in Java Software Systems.	2019	[51]
S30	Augmenting Java Method Comments Generation with Context Information based on Neural Networks.	2019	[52]
S31	Improving Source Code Lexicon via Traceability and Information Retrieval.	2011	[53]
S32	Detecting API Documentation Errors.	2013	[54]
S33	Analyzing Code Comments to Boost Program Comprehension	2018	[55]
S34	Recommending Reference API Documentation.	2015	[56]
S35	Some Structural Measures of API Usability	2015	[57]
S36	An Empirical Study of the Textual Similarity between Source Code and Source Code Summaries.	2016	[58]
S37	Linguistic Antipatterns: What They are and How Developers Perceive Them.	2016	[59]
S38	Coherence of Comments and Method Implementations: A Dataset and An Empirical Investigation	2016	[60]
S39	A Comprehensive Model for Code Readability	2018	[61]
S40	Automatic Detection of Outdated Comments During Code Changes	2018	[62]
S41	Classifying Python Code Comments Based on Supervised Learning	2018	[63]
S42	Investigating Type Declaration Mismatches in Python	2018	[64]
S43	The Exception Handling Riddle: An Empirical Study on the Android API.	2018	[65]
S45	Migrating Deprecated API to Documented Replacement: Patterns and Tool	2019	[66]
S46	A Topic Modeling Approach To Evaluate The Comments Consistency To Source Code	2020	[67]
S47	Comparing Identifiers and Comments in Engineered and Non-Engineered Code: A Large-Scale Empirical Study	2020	[68]

Table 4: RQ2 QAs mentioned by Zhi *et al.* (highlighted in bold) and other works ¹

Quality Attribute (QA)	Synonyms	Description
<i>QAs mentioned by Zhi et al.</i>		
Accessibility	availability, information hiding, easiness to find	whether comment content can be accessed or retrieved by developers or not
Readability	clarity	the extent to which comments can be easily read by other readers
Spelling and grammar	natural language quality	grammatical aspect of the comment content
Trustworthiness		the extent to which developers perceive the comment as trustworthy
Author-related		identity of the author who wrote the comment
Correctness		whether the information in the comment is correct or not
Completeness	adequacy	how complete the comment content is to support development and maintenance tasks or whether there is missing information in comments or not
Similarity	uniqueness, duplication	how similar the comment is to other code documents or code
Consistency	uniformity, integrity	the extent to which the comment content is consistent with other documents or code
Traceability		the extent to which any modification in the comment can be traced, including who performed it
Up-to-datedness		how the comment is kept up-to-date with software evolution
Accuracy	preciseness	accuracy or preciseness of the comment content. If the documentation is too abstract or vague and does not present concrete examples, then it can seem imprecise.
Information organization		how the information inside a comment is organized in comments
Format	including visual models, use of examples	quality of documents in terms of writing style, description perspective, use of diagrams or examples, spatial arrangement, etc.
<i>QAs mentioned by other works</i>		
Coherence		how comment and code are related to each other, <i>e.g.</i> , method comment should be related to the method name(S02, S38)
Conciseness		the extent to which comments are not verbose and do not contain unnecessary information (S23, S30, S47)
Content relevance		how relevant the comment or part of the comment content is to a particular purpose (documentation, communication) (S01, S03, S29, S41, S47)
Maintainability		the extent to which comments are maintainable (S15-S17,S20-S21)
Understandability		the extent to which comments contribute to understanding the system (S19, S23)
Usability	Usefulness	to which extent the comment can be used by readers to achieve their objectives (S02, S16, S34, S35)
Documentation technology		whether the technology to write, generate, store documentation is current or not
Internationalization		the extent to which comments are correctly translated in other languages (S16)
Other		the study does not mention any QA and cannot be mapped to any of the above attributes

Zhi *et al.* in their work [18], and are highlighted by the bold text compared to QAs mentioned in other works. As Zhi *et al.* considered various types of documentation, such as requirement and architectural documents, not all attributes fit exactly into our study. For instance, the category “Format” includes the format of the documentation (*e.g.*, UML, flow chart) in addition to the other aspects such as writing style of the document, use of diagrams *etc.* Although the format of the documentation is not applicable in our case due to our comment-specific interest, we keep other applicable aspects (writing style, use of diagram) of this QA. In addition to their QAs, we include any additional attribute mentioned in our set of relevant papers. If a study uses different terminology but similar meaning to QAs in our list, we map such QAs to our list and update the list of possible synonyms as shown in the column *Synonyms* in Table 4. In case we cannot map a study to the existing QAs, we map it to the *Other* category.

For the cases where the studies do not mention any specific QA and mention comment quality analysis in general, we map the study to the list of existing QAs or classify it as *Other* based on their goal behind the quality analysis. For example, Pascarella *et al.* identify various information types in comments to support developers in easily finding relevant information for code comprehension tasks and to improve the comment quality assessment [13]. They do not mention any specific QA, but based on their study goal of finding relevant information easily, we map their study to the *content relevance* QA. Similarly, we map other comment classification studies such as S06, S29, S33, and S41 to the *content relevance* attribute. At the same time, the studies on linguistic anti-patterns (LAs) are mapped to the *consistency* attribute, given that LAs are practices that lead to lexical inconsistencies among code elements, or between code and associated comments [59, 34, 35]. Additionally, the studies that mention the negation of the QAs such as *inconsistency*, *incorrectness*, or *incompleteness* are mapped to their antonyms as *consistency*, *correctness*, or *completeness*, respectively to prevent duplication.

RQ3 (Which tools and techniques do researchers use to as-

sess comment QAs?) concerns various methods researchers use or propose to assess comment QAs, for instance, whether they use machine-learning based methods to assess comment quality or not.

- **Technique type.** This identifies whether the technique used to assess a QA is based on natural language processing (NLP), heuristics, static analysis, metrics, machine-learning (ML), or deep neural network (DNN) approaches. The rationale is to identify which QAs are often assessed manually or using a specific automated approach. For instance, if the study uses specific heuristics related to the programming environment to assess a QA, it is classified as *heuristic-based* technique, if it uses abstract syntax tree (AST) based static analysis approaches, then it is assigned to *static analysis*, and if it uses machine-learning or deep-learning-based techniques (including any or both of the supervised or unsupervised learning algorithms), then it is classified as *ML-based*, or *DNN-based* respectively. A study can use mixed techniques to assess a specific QA and thus can be assigned to multiple techniques for the corresponding QA. We often find cases where the studies do not use any automated technique to measure a QA and instead ask other developers to assess it manually, so we put such cases into the *manual assessment* category. In case the study mentions a different technique, we extend the dimension values.

- **Metrics or tools.** This further elaborates specific metrics, or tools the studies propose or use to assess a QA. A study can use an existing metric or can propose a new one. Similarly, one metric can be used to assess multiple QAs. We identify such metrics to highlight popular metrics amongst researchers.

RQ4 (What kinds of contribution do studies often make?) captures the nature of the study and the type of contribution researchers use or propose to assess comment quality. We first identify the nature of research of a study and then identify the type of contribution it provides. This can reflect the kind of re-

search often conducted to assess comment quality and the kind of contribution they make to support developers in assessing comment quality, for instance, what kind of solutions the *Solution Proposal* research often propose, such as a method, metric, model, or tool.

To capture this information, we formulate the following dimensions:

- *Research type*. This identifies the nature of the research approach used in the studies, such as empirical, validation, evaluation, solution proposal, philosophical, opinion, or experience paper [69, 70]. The dimension values are described in detail in Table 5.
- *Paper contribution*. This dimension describes the type of contribution the study provides in terms of a method/technique, tool, process, model, metric, survey, or empirical results [70]. The dimension values are described in detail in Table 5. If we cannot categorize it into any of these, we mark it “Other”.
- *Tool availability*. This reflects whether the tool proposed in the study is accessible or not at the time of conducting our study. González *et al.* identified the reproducibility aspects characterizing empirical software engineering studies [71] in which availability of the artifact (the tool proposed in the study, or the dataset used to conduct the study) is shown as an important aspect to facilitate the replication and extension of the study. Therefore, we record the availability of the proposed tool in this dimension and the availability of the dataset in the following dimension.
- *Dataset availability*. This reflects if the dataset used in the empirical study is accessible or not.

RQ5 (How do researchers evaluate their comment quality assessment studies?) concerns how various kinds of research (*Research type* dimension described in the previous RQ), and various kinds of contribution (*Paper contribution* dimension) are evaluated in the studies. For example, it helps us to observe that if a study proposes a new method/technique to assess com-

ments, then the authors also conduct an experiment on open-source projects to validate the contribution, or they consult the project developers, or both. We capture the type of evaluation in the *Evaluation type* dimension, and its purpose in *Evaluation purpose*. The rationale behind capturing this information is to identify the shortcomings in their evaluations, *e.g.*, how often the studies proposing a tool are validated with practitioners.

- *Evaluation type*. It states the type of evaluation the studies conduct to validate their approaches, such as conducting an experiment on open-source projects (*Experiment*), or surveying students, practitioners, or both. For the automated approaches, we consider various performance metrics, also known as Information Retrieval (IR) metrics, that are used to assess the machine/deep learning-based models, such as Precision, Recall, F1 Measure, or Accuracy under the *performance metrics*. In case the approach is validated by the authors of the work, we identify the evaluation type as *Authors of the work*.
- *Evaluation purpose*. It states the motivation of evaluation by authors such as evaluate the functionality, efficiency, applicability, usability, accuracy, comment quality in general, or importance of attributes.

3. Results

As mentioned in subsection 2.2.5, we analyze 47 relevant papers in total. Before answering our four RQs, we present a brief overview of the metadata (publishing venues) of the papers.

Table 2 highlights the publication venues of these papers. Most studies were published in top-tier software engineering conferences (*e.g.*, ICSE) and journals, especially the ones with a focus on empirical studies (*e.g.*, EMSE). This means that the SE community agrees that assessing comment quality is an important topic deserving of research effort. Figure 2 shows the paper distribution over the past decade, indicating a clear trend of increasing interest of the SE research community in comment quality assessment. Figure 3 shows the author distribu-

Table 5: Type of research approach studies use and type of contributions studies make ¹

Dimension	Category	Description
Research type	Empirical	This research task focuses on understanding and highlighting various problems by analyzing relevant projects, or surveying developers. These papers often provide empirical insights rather than a concrete technique.
	Validation	This research task focus on investigating the properties of a technique that is novel and is not yet implemented in practice, <i>e.g.</i> , techniques used for mathematical analysis or lab experimentation
	Evaluation	The paper investigates the techniques that are implemented in practice and their evaluation is conducted to show the results of the implementation in terms of its pros and cons and thus help researchers in improving the technique.
	Solution Proposal	The paper proposes a novel or a significant extension of an existing technique for a problem and describes its applicability, intended use, components, and how the components fit together using a small example or argumentation.
	Philosophical	These papers present a new view to look at the existing problems by proposing a taxonomy or a conceptual framework, <i>e.g.</i> , developing a new language or framework to describe the observations is a philosophical activity.
	Opinion	These papers describe the author’s opinion in terms of how things should be done, or if a certain technique is good or bad. They do not rely on research methodologies and related work.
	Experience	These papers explain the personal experience of a practitioner in using a certain technique to show how something has been done in practice. They do not propose a new technique and are not scientific experiments.
Contribution type	Empirical	The paper provides empirical results based on analyzing relevant projects to understand and highlights the problems related to comment quality.
	Method/technique	The paper provides a novel or significant extension of an existing approach.
	Model	Provides a taxonomy to describe their observations or an automated model based on machine/deep learning.
	Metric	Provides a new metric to assess specific aspects of comments.
	Survey	Conducts survey to understand a specific problem and contribute insights from developers.
	Tool	Develops a tool to analyze comments.

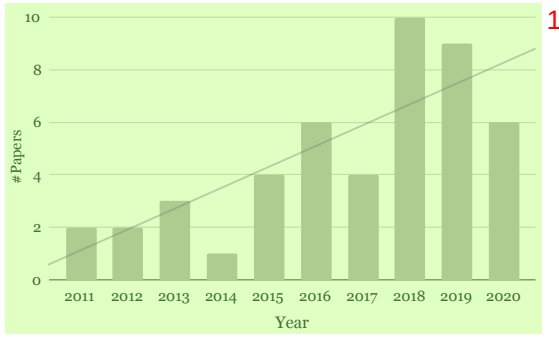


Figure 2: Relevant papers by years

tion of the selected papers by the institution. For the timeline 1971-2011, we rely on the geographical statistics data from the replication package of our reference study by Zhi *et al.* [18], while for the period 2011-2021, and we collect these statistics as follows. For each paper, the primary affiliations of all authors are taken into account. If people from different countries co-authored a paper, we calculate the proportion of a country’s contribution for each paper so that each paper gets a total score of one to avoid over-representing papers. For example, if five authors of a paper belong to Switzerland and one belongs to Spain, we assign 5/6 score for Switzerland and 1/6 for Spain for the paper. Comparison with the previous data allows us to see the evolution of the field, with more even distribution of researchers nowadays and (unsurprising) rise of contributions from southeast Asia, specifically from China.

Finding 1. The trend of analyzing comment quality has increased in the last decade (2011-2020), in part due to more researchers from southeast Asia working on the topic.

3.1. RQ₁: What types of comments do researchers focus on when assessing comment quality?

To describe the rationale behind code implementation, various programming languages use source code comments. Our results show that researchers focus more on some programming languages compared to others as shown in Figure 4. This plot highlights the types of comments on the y-axis; each stack in the bar shows the ratio of the studies belonging to a particular language. For instance, the majority (87%) of the studies focus

on code comments from Java, whereas only 15% of the studies focus on code comments from Python, and 10% of them focus on C# and C++. These results are in contrast to popular languages indicated by various developer boards, such as GitHub, Stack Overflow, or TIOBE. For instance, the TIOBE index show Python and C languages more popular than Java. Similarly, the developer survey of 2019 and 2020 by Stack Overflow show that Java stands fifth after JavaScript, HTML/CSS, SQL, and Python among the most commonly used programming languages. We find only one study (S44) that seems to address the comment quality aspect in JavaScript. Given the emerging trend of studies leveraging natural-language information in JavaScript code, more research about comment quality may be needed in this environment. It indicates that researchers need to analyze comments of other languages to verify their proposed approaches and support developers of other languages.

Finding 2. 87% of the studies analyze comments from Java while other languages have not yet received enough attention from the research community.

As code comments play an important role in describing the rationale behind source code, various programming languages use different types of comments to describe code at various abstraction levels. For example, Java class comments should present high-level information about the class, while method comments should present implementation-level details. We find that half of the studies (51% of the studies) focus on all types of comments whereas the other half focus on specific types of comments, such as inline, method, or TODO comments. However, we also see in Figure 4 that studies frequently focus on method comments and API documentation. This proves the effort the research community is putting into improving API quality. While some attention is given to often overlooked kinds of comments, such as license comments (S28,S33), TODO comments (S14), inline comments (S17), and deprecation comments (S45), no relevant paper seems to

⁷<https://www.tiobe.com/tiobe-index/> verified on Sep, 2021

⁸<https://insights.stackoverflow.com/survey/2020>

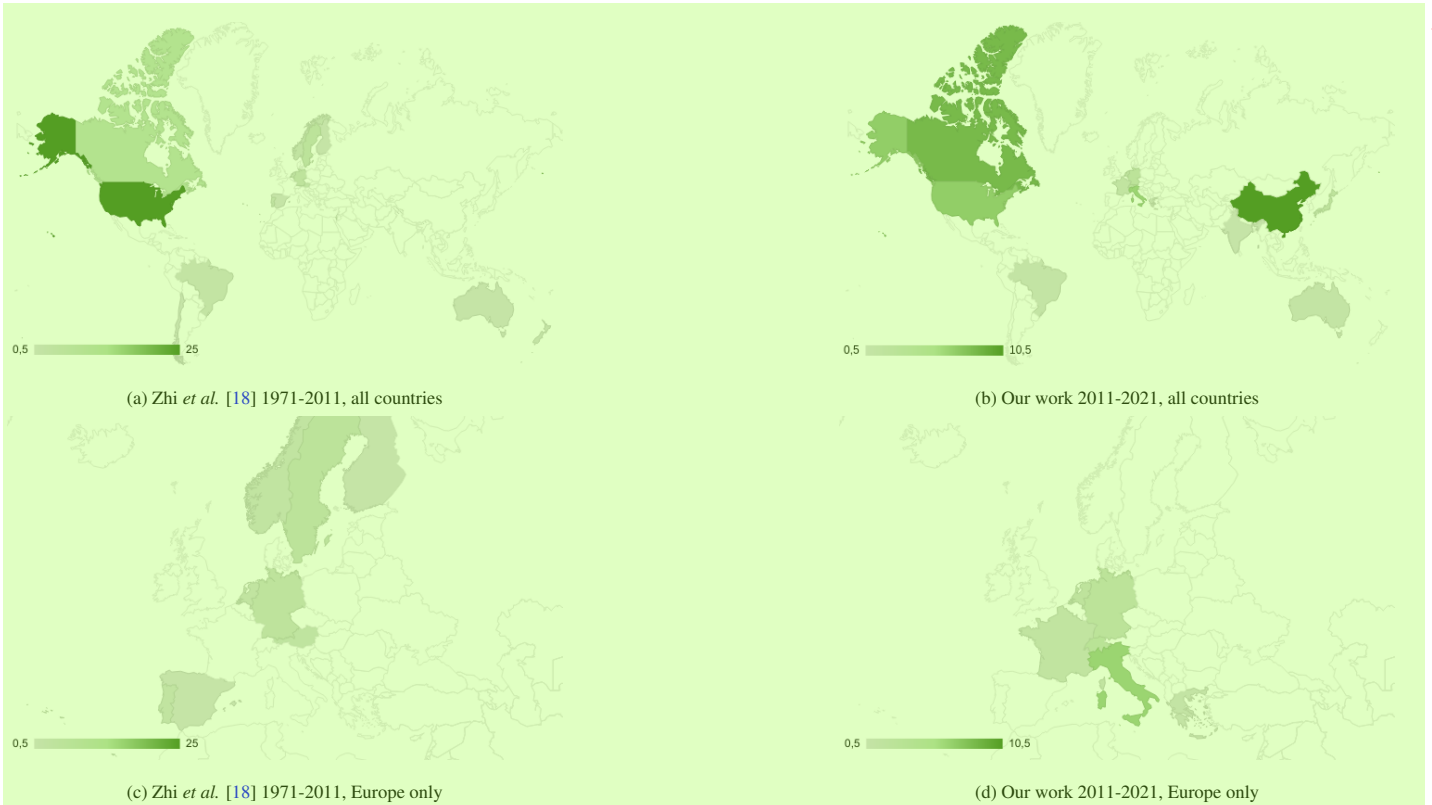


Figure 3: Relevant papers by countries

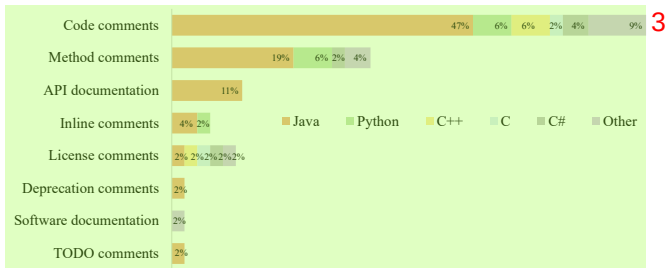


Figure 4: Types of comments per programming language

Finding 3. Even though 50% of the studies analyze all types of code comments, the rest focus on studying a specific type of comments such as method comments, or API comments, indicating research interest in leveraging a particular type of comment for specific development tasks.

Previous work by Zhi *et al.* showed that a majority of studies analyze just one type of system [18]. In contrast, our findings suggest that the trend of analyzing comments of multiple languages and systems is increasing. For example, 80% of the studies analyzing comments from Python and all studies analyzing comments from C++ also analyze comments from Java. Only Pascarella *et al.* (S42) and Zhang *et al.* (S41) focus solely on Python [64, 63]. However, Zhang *et al.* (S41) perform the comment analysis work in Python based on the Java study (S29) by Pascarella *et al.* [63, 13]. Such trends also reflect the increasing use of polyglot environments in software development [77]. The “Other” label in Figure 4 comprises language-agnostic studies, *e.g.*, S16 or the studies considering less popular languages, *e.g.*, S28 focuses on COBOL. We find

focus specifically on the quality of *class* or *package* comments. Recently Rani *et al.* studied the characteristics of class comments of Smalltalk in the Pharo environment⁹ and highlighted the contexts they differ from Java and Python class comments, and why the existing approaches (based on Java, or Python) need heavy adaption for Smalltalk comments [75, 76]. This may encourage more research in that direction, possibly for other programming languages.

⁹<https://pharo.org/>

only one study (S44) that analyzes comments of six programming languages *et al.* [26].

Finding 4. The trend of analyzing multiple software systems of a programming language, or of several languages, shows the increasing use of polyglot environments in software projects.

3.2. RQ₂: Which QAs are used to assess code comments?

To characterize the attention that the relevant studies reserve to each QA over the past decade, Figure 5 shows all the QAs on the y-axis and the corresponding years on the x-axis. Each bubble in the plot indicates both the number of papers by the size of the bubble and IDs of the studies. Comparing the y-axis with the QAs in Table 4 demonstrates that our analysis finds new QAs with respect to the previous work of Zhi *et al.* The additional QAs are: *usefulness*, *use of examples*, *usability*, *references*, *preciseness*, *natural language quality*, *maintainability*, *visual models*, *internationalization*, *documentation technology*, *content relevance*, *conciseness*, *coherence*, and *availability*. However, not all QAs reported by Zhi *et al.* for software documentation quality (highlighted in bold in Table 4) are used in comment quality assessment. In particular, we find no mention of *trustworthiness*, and *similarity* QAs even though previous works have highlighted the importance of both QAs to have high-quality documentation [78, 79, 80]. Also, Maalej *et al.* showed in their study that developers trust code comments more than other kinds of software documentation [81], indicating the need to develop approaches to assess the trustworthiness of comments.

Finding 5. Compared to the previous work by Zhi *et al.*, we find 10 additional QAs researchers use to assess code comment quality.

Although several QAs received attention in 2013, the detailed analysis shows that there were mainly two studies (S02, S03) covering several QAs. There is only one study published in 2014 (S05), while 2015 sees the first studies focusing on assessing comment quality. One in particular, S26, attempts to cover multiple QAs. The plot also shows which QAs receive the most attention. A few QAs such as *completeness*, *accuracy*, *content relevance*, *readability* are often investigated. The QA

consistency is by far the one that receives constant and consistent attention across the years, with several in 2017 (S07, S08, S09, S29) and 2018 (S10, S11, S39, S42, S43). Indeed, the problem of inconsistency has been studied from multiple points of view, such as inconsistency between code and comments that may emerge after code refactoring (S07), or the inconsistencies revealed by so-called *linguistic antipatterns* (S11, S37). Unsurprisingly, the plot shows that *up-to-dateness* increasingly has received attention in the last three years of the decade, given that comments that are not updated together with code are also a cause of inconsistency (S15, S16).

A few attributes are rarely investigated, for instance the QAs investigated only by at most two studies over the past decade are *format*, *understandability*, *spelling & grammar*, *organization*, *internationalization*, *documentation technology*, *coherence*, *conciseness*, *author related* and *accessibility*. More research would be needed to assess whether such attributes are intrinsically less important than others for comments according to practitioners.

Finding 6. While QAs such as *consistency* and *completeness* are frequently used to assess comment quality, others are rarely investigated, such as *conciseness* and *coherence*.

Another aspect to analyze is whether researchers perceive the QAs as being the same or not. For example, do all studies mean the same by *consistency*, *conciseness*, *accuracy* of comments? We therefore collect the definition of each QA considered in the study. We find that for various QAs researchers refer to the same QA but using different terminology. We map such cases to the *Synonyms* column presented in Table 4. From this analysis we find that not all studies precisely define the QAs, or they refer to their existing definitions while evaluating comments using them. For instance, the studies (S01, S04, S13, S17, S20, S29, S41) do not mention the specific QAs or their definition. We put such studies, classifying comment content with the aim to improve comment quality, under *content relevance*. On the other hand, in some studies researchers mention the QAs but not their definition. For instance, S26 refers to various existing studies for the QA definitions but which QA definition is ex-

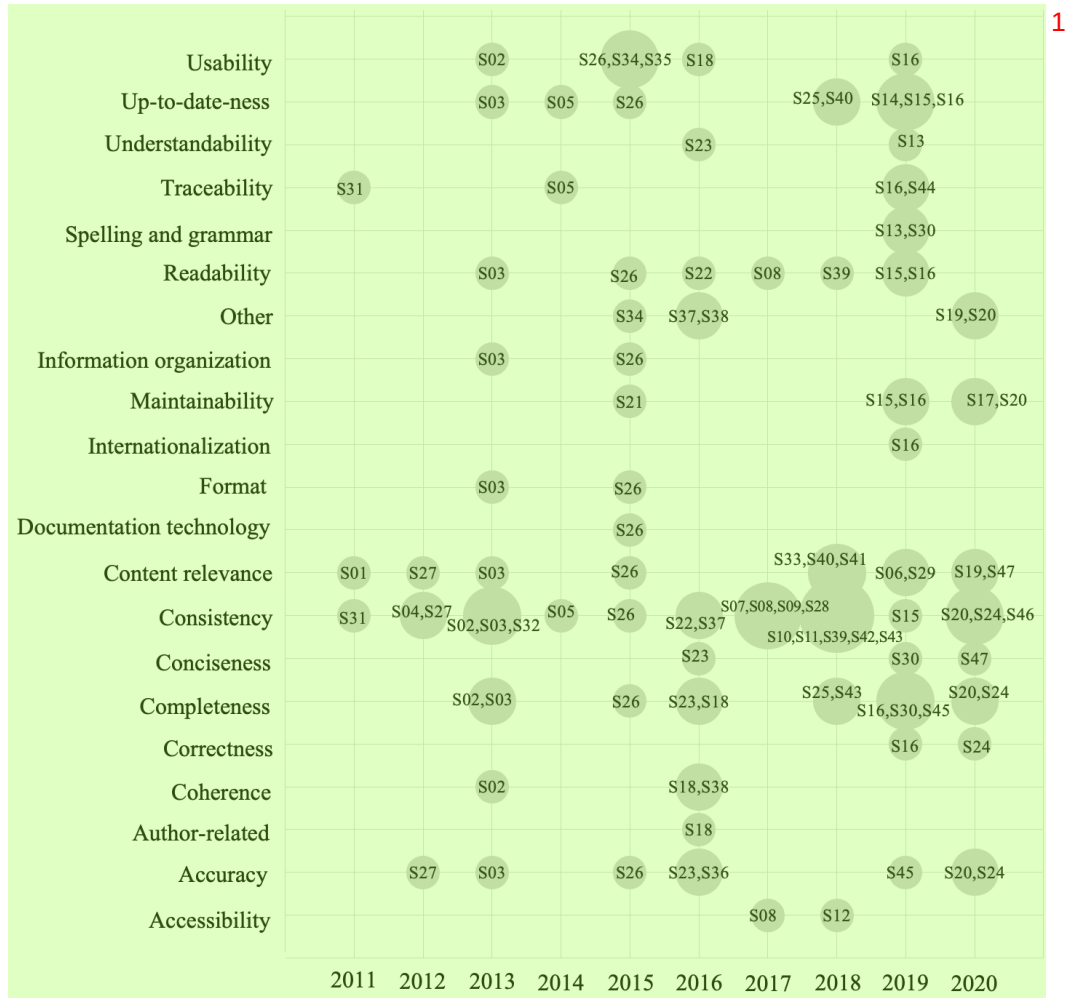


Figure 5: Frequency of various comment quality QAs over year

tracted from which study is not very clear. Lack of precise definitions of QAs or having different definitions for the same QAs can create confusion among developers and researchers while assessing comment quality. Future work needs to pay attention to either refer to the existing standard definition of a QA or define it clearly in the study to ensure the consistency and awareness across developer and scientific communities. In this study, we focus on identifying the mention of QAs and their definition if given, and not on comparing and standardizing their definition. Such work would require not only the existing definitions available in the literature for QAs but also collecting how researchers use them in practice, and what developers perceive from each QA for source code comments, which is out of scope for this work. However, we provide the list of QAs researchers use for comment quality assessment to facilitate future work in

mapping their definition and standardizing them for code comments.

Although each QA has its own importance and role in comment quality, they are not measured in a mutually exclusive way. We find cases where a specific QA is measured by measuring another QA. For example, *accuracy* is measured by measuring the *correctness* and *completeness* of comment, such as “the documentation is incorrect or incomplete and therefore no longer accurate documentation of an API.” (S24) Similarly, *up-to-date-ness* is measured through *consistency* of comments (S40) or *consistency* is evaluated and improved using *traceability* (S31). This indicates the dependency of various QAs on each other, and improving one aspect of comments can automatically improve other related aspects. However, which techniques are used to measure which QAs is not yet known.

Finding 7. Many studies miss a clear definition of the QAs they use in their studies. This poses various challenges for developers and researchers, *e.g.*, understanding what a specific QA means, mapping a QA to other similar QAs, and adapting the approaches to assess the QA to a certain programming environment.

3.3. RQ₃: Which tools and techniques do researchers use to assess comment QAs?

With respect to each QA, we first identify which techniques have been used to measure them. We use the dimension *Technique type* to capture the type of techniques. Figure 6 shows that the majority of the QAs are measured by asking developers to manually assess it (*manual assessment*). For instance, QAs such as *coherence*, *format*, *organization*, *understandability*, and *usability* are often assessed manually. This indicates the need and opportunities to automate the measurement of such QAs.

A significant number of studies experimented with various automated approaches based on machine or deep learning approaches, but they focus on specific QAs and miss other QAs such as *natural language quality*, *conciseness*, *correctness*, *traceability*, *coherence* *etc.* Similarly, another significant portion of studies uses heuristic-based approaches to measure various QAs. The limitation of such heuristic-based approaches is their applicability to other software systems and programming languages. More studies are required to verify the generalizability of such approaches.

Finding 8. *Manual assessment* is still the most frequently-used technique to measure various QAs. Machine learning based techniques are the preferred automated approach to assess QAs, but the majority of them focus on specific QAs, such as *consistency*, *content relevance*, and *up-to-dateness*, while ignoring other QAs.

We find that the majority of the machine learning-based approaches are supervised ML approaches. These approaches require labeling the data and are therefore expensive in terms of time and effort. To avoid the longer training time and memory consumption of ML strategies, Kallis *et al.* used *fastText* to classify the issues reports on GitHub [82]. The *fastText* tool uses linear models and has achieved comparable results in classification to various deep-learning based approaches. A recent

study by Minaee *et al.* shows that deep learning-based approaches surpassed common machine learning-based models in various text analysis areas, such as news categorization and sentiment analysis [83]. We also find some studies that use deep learning-based techniques partly (S06, S13, S20) along with machine learning techniques for a few QAs, such as assessing *conciseness*, *spelling and grammar*, and *completeness*. However, there are still many QAs that are assessed manually and require considerable effort to support developers in automatically assessing comment quality.

Finding 9. In the case of automated approaches to assess various QAs of comments, we observe that deep-learning based approaches are not yet explored even though various studies showed that they surpassed ML-based approaches in text analysis areas.

We see that machine learning-based approaches are used more often than deep-learning approaches, but whether it is due to their high accuracy, easy interpretation, or need for a small dataset is unclear and requires further investigation.

In addition to identifying general techniques, we collect which metrics and tools have been used to measure various QAs. Table 6 shows various QAs in the column *QAs*, and metrics and tools used for each QA in the column *Metrics*, and *Tools* respectively. The description of the collected metrics is presented in Table 7. We can see that out of 21, only 10 QAs have metrics defined for them.

A software metric is a function that takes some software data as input and provides a numerical value as an output. The output provides the degree to which the software possesses a certain attribute affecting its quality [84]. To limit the incorrect interpretation of the metric, threshold values are defined. However, the threshold value may change according to the type of comments analyzed, and the interpretation of the metric output may vary in turn. We report threshold values, if present, for the collected metrics.

For *readability* QA, researchers were often found to be using the same metric (S08, S22, S39). As developers spend significant amount of time reading code, including comments, having readable comment can help them in understanding code eas-

Table 6: Metrics and tools used for various quality attributes. 1

Note: the description of each metric is given in Table 7 2

QAs	Metrics	Tools
Accessibility	S08: Accessibility_1, Accessibility_2	S12: Text2KnowledgeGraph
Readability	S08: Readability_1 S22: Readability_1 S39: Readability_1	
Spelling and Grammar	S13: SpellGrammar_1	
Correctness		S24: Drone
Completeness	S18: Completeness_1, Author_1 S02: Completeness_2 S43: Completeness_3	S24: Drone S45: DAAMT
Consistency	S08: Consistency_1 S22: Consistency_1 S39: Consistency_1 S46: Consistency_2 S28: LicenseConsistency_1	S07: Fraco S06: RecoDoc, AdDoc S24: Drone S31: Coconut S09: Zhou et. al S37: LAPD S42: PyID
Traceability		S06: RecoDoc S31: Coconut
Up-to-datedness		S06: RecoDoc, AdDoc S14: Trigit
Accuracy	S36: Accuracy_1	S24: Drone S45: DAAMT
Coherence	S02: Coherence_1, Coherence_2 S18: Coherence_3 S38: Coherence_4	
Maintainability	S21: Coherence_4	S17: Pham et. al.
Understandability	S13: Understandability_1	
Usability	S35: Usability_1	S34: Krec

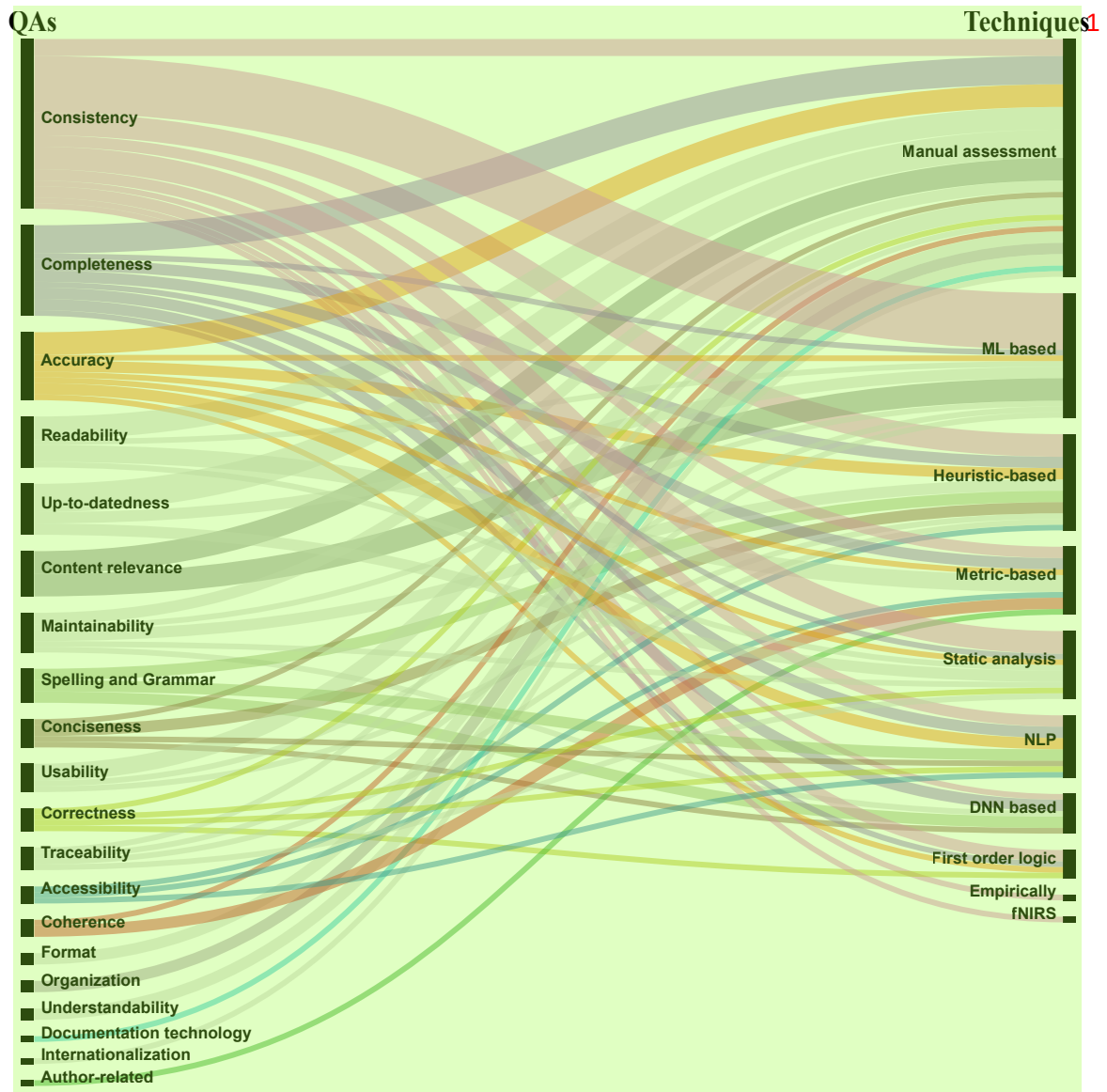


Figure 6: Types of techniques used to analyze various QAs

ier. Yet readability remains a subjective concept. Several studies, such as S08, S22, S39 identified various syntactic and textual features for source code and comments. However, in context of code comments, they focus on the Flesch-Kincaid index method, which is typically used to assess readability of natural language text. Since comments often consist of a mix of source code and natural language text, such methods can have disadvantages. For example, developers can refer to the same code concept differently in comments, and they can structure their information differently. Thus, formulating metrics that consider the special context of code comments can improve the the as-

essment of readability of comments.

Another popular metric is *Consistency_1* used for assessing consistency between comments and code (S08, S22, S39). This metric measures the overlap between the terms of method comments and method body. These studies assume that the higher the overlap, better the readability of that code. Similarly, metrics (*coherence_1* , *coherence_3* , *coherence_4*) used for measuring the *coherence* QA suggest higher overlap between comments and code. However, having too many overlapping words can defy the purpose of comments and can lead to redundant comments. Using such metrics, a comment containing only

Table 7: Description of each metric listed in Table 6 1

Metrics	Description
Accessibility_1	MIDQ: (Documentable items of a method + readability of comments)/2. [S08]
Accessibility_2	AEDQ: Identify all Stack overflow discussions that have "how to" words and the class name in the title. [S08]
Accuracy_1	Short Text Semantic Similarity (STSS): the intersection of keywords between summaries and source code, STASIS (word semantic similarity, sentence semantic similarity, and word order similarity), LSS (Lightweight Semantic Similarity). [S36]
Author_1	A class comment should contain authorship. Check the presence and absence of the @author tag with the following name. [S18]
Coherence_1	The similarity between words from method comments and method names where similarity is computed using Levenshtein distance. The value should be between 0 and 0.5 to have a coherent comment. [S02]
Coherence_2	The length of comments should be between 2 words to 30 words. [S02]
Coherence_3	Percentage of the number of class or method's words contained in the class or method comments divided by the total class or method's words. The value should be above or equal to 0.5. [S18]
Coherence_4	There is coherence between the comment and the implementation of a method when they have a high lexical similarity, where lexical similarity is computed using cosine similarity. [S38]
Completeness_1	A class comment should contain a description and authorship. A method should contain comments if it is complex (more than three method invocation) and have 30 LOC. [S18]
Completeness_2	How many of the public classes, types, and methods have a comment preceding them. [S02]
Completeness_3	Exceptions that are present in App Programs, Crashes, and API source code but not in API reference documentation. [S43]
Consistency_1	The overlap between the terms used in a method comment and the terms used in the method body. They correlate a higher value of CIC with a higher readability level of that code. [S08, S22, S39]
Consistency_2	The Kullback-Leibler divergence is a measure that finds the difference between two probability distributions. [S46]
LicenseConsistency_1	Two similar source code files have different licenses. Find the number of files in a group, number of different licenses in the group, number of files with an unknown license in the group, number of files without any license in the group, and number of licenses in the GPL family. [S28]
Readability_1	Flesch reading-ease test. [S08, S22, S39]
SpellGrammar_1	The sentence has no subject or predicate, or has incomplete punctuations (e.g., the right parenthesis is missing). [S13]
Understandability_1	Remove a sentence if it is incomplete, contains code elements, is a question, or it mentions the concept in its subordinate clauses. [S13]
Usability_1	ADI: number of words in the method comments. The threshold is decided based on the simple average of the ADI for all method declaration. [S35]

rationale information about a method or class might be qualified as an incoherent or inconsistent comment whereas such comments can be very helpful in providing additional important information. Although metrics can help developers easily estimate the quality of comments, their sensitivity towards various QAs can degrade comment quality overall. More research is required to know the implication of given metrics on various QAs or combinations of QAs.

Finding 10. Nearly 25% of the studies use metric-based methods to measure comment quality. However, the metrics are defined or used for only 10 QAs out of 21 QAs.

3.4. RQ₄: What kinds of contribution do studies often make? 4

Research types. As a typical development cycle can contain various research tasks, such as investigation of a problem, or validation of a solution, we collect which types of research are performed for the comment quality assessment domain, and what kinds of solutions researchers often contribute. We categorize the papers according to the *research type* dimension and show its results in Figure 7. The results show that the studies often conduct validation research (investigating the properties of a solution) followed by the solution proposal (offering a proof-of-concept method or technique). However, very few

Research type	Paper contribution					
	Empirical results	Method/Technique	Metric	Model	Survey	Tool
Empirical study	S11 S15 S16 S26	S44 S47	S36	S27 S37	S01 S03 S25	
Evaluation		S05 S29 S21 S43		S22	S08	
Replication study	S19					
Solution proposal		S13 S23 S28 S30 S33 S41 S46		S39 S48 S40		S17 S31 S32 S34
Validation		S02 S04 S07 S09 S10 S12 S42 S45	S35 S18	S06 S29 S38		S14 S24

Figure 7: Types of contribution for each research type

Paper contribution	Evaluation type					
	Authors of the work	Experiment	Performance metrics	Survey practitioners	Survey practitioners and students	Survey students
Empirical results	S15 S16	S11		S26	S19	
Method/Technique	S30 S42 S44 S47	S13 S28 S43 S45 S46	S02 S04 S05 S07 S09 S13 S20 S41 S42	S02 S05 S21	S20 S23 S43	S10 S12
Metric	S36	S35			S18	
Model	S27 S38	S06 S27 S37 S38 S39	S06 S22 S29 S33 S40		S37	S40
Survey				S01 S03 S25	S08	
Tool		S32 S34	S17 S24		S14 S34	S24 S31

Figure 8: Types of evaluation for each paper contribution type

studies focus on evaluation research (investigating the problem or a technique implementation in practice). We find only one study performing a replication study (S19). Given the importance of research replicability in any field, future work needs to focus more on evaluating the proposed solution and testing their replicability in this domain.

Paper contribution types. By categorizing the papers according to the *paper contribution* definition, Figure 7 and Figure 8 show that over 44% of papers propose an approach (method/technique) to assess code comments. A large part (75%) of them are heuristics-based approaches, e.g., Zhou *et al.* and Wang *et al.* present such NLP based heuristics (S9, S13). A few approaches rely on manual assessments. As an example, consider how taxonomies assessing comment quality have emerged [14, 39]. Models are the second contribution by frequency, which makes sense considering the increasing trend

of leveraging machine learning during the considered decade: 60% of the relevant papers proposing models are based on such approaches. The label *Empirical results* comprises studies which mainly offer insights through authors’ observations (e.g., S11, S15, S16, S19, S26). Finally, given the important role that metrics have in software engineering [85, 86], it is valuable to look into metrics that are proposed or used to assess code comment quality as well. For example, three studies (S18, S35, and S36) contribute metrics for *completeness*, *accuracy*, or *coherence* whereas other studies use existing established metrics, e.g., S08, S22, or S39 compute the *readability* of comments using the metric named the Flesch-Kincaid index.

Tool availability. Previous work indicates the developers’ effort in seeking tools to assess documentation quality, and highlights the lack of such tools [39]. In our study, we find that 32% of the studies propose tools to assess specific QAs, mainly

for detecting inconsistencies between code and comments. Of these studies proposing tools, 60% provide a link to them. The lack of a direct link in the remaining 40% can hinder the reproducibility of such studies.

Dataset availability. In terms of dataset availability, 49% of the studies provide a link to a replication package. Of the remaining papers, some provide a link to the case studies they analyze (typically open-source projects) [28], build on previously existing datasets [61], or mention the reasons why they could not provide a dataset. For instance, Garousi *et al.* indicated the company policy as a reason to not to share the analyzed documentation in their case study [48].

Finding 11. Nearly 50% of the studies still are lacking on the *reproducibility* dimension, with their respective dataset or tool often not publicly accessible.

3.5. RQ5: How do researchers evaluate their comment quality assessment studies?

Figure 8 shows how authors evaluate their contributions. We see that code comment assessment studies generally lack a systematic evaluation, surveying only students, or conducting case studies on specific projects only. Most of the time, an experiment is conducted without assessing the results through any kind of external expertise judgment. Hence, only 30% of the relevant studies survey practitioners to evaluate their approach. This tendency leads to several disadvantages. First, it is difficult to assess the extent to which a certain approach may overfit specific case studies while overlooking others. Second, approaches may be unaware of the real needs and interests of project developers. Finally, the approaches may tend to focus too little on real-world software projects (such as large software products evolving at a fast pace in industrial environments). Similarly, when a new *method* or *technique* or comment classification *model* is proposed, it is often assessed based on conventional performance metrics, such as Precision, Recall, or F1 (S02, S04, S07, S29, S41 *etc.*) and rarely are the results verified in an industry setting or with practitioners.

Finding 12. Many code comment assessment studies still lack systematic industrial evaluations for their proposed approaches, such as evaluating the *metric*, *model*, or *method/technique* with practitioners.

4. Discussion

Below we detail our observations about state of the art in comment quality analysis together with implications and suggestions for future research.

Comment Types. The analysis of the comment quality assessment studies in the last decade shows that the trend of analyzing comments from multiple languages and systems is increasing compared to the previous decade where a majority of the studies focus on one system [18]. It reflects the increasing use of polyglot environments in software development [77]. Additionally, while in the past researchers focused on the quality of code comments in general terms, there is a new trend of studies that narrow their research investigation to particular comment types (methods, TODOs, deprecation, inline comments), indicating the increasing interest of researchers in supporting developers in providing a particular type of information for program comprehension and maintenance tasks.

Emerging QAs. Our analysis of the last decade of studies on code comment assessment shows that new QAs (*coherence*, *conciseness*, *maintainability*, *understandability etc.*), which were not identified in previous work [18], are now being investigated and explored by researchers. This change can be explained by the fact that while in the past researchers focused on the quality of code comments in general terms, in the last decade there has been a new trend of studies that narrow their research investigation to specific comment types (methods, TODOs, deprecation, inline comments) and related QAs.

Mapping QAs. As a consequence of this shift of focus towards specific comment types, the same QAs used in prior studies can assume different definition nuances, depending on the kind of comments considered. For instance, let us consider how the QA *up-to-dateness*, referred to in studies on code-comment *inconsistency*, assumes a different interpretation in the context of

TODO comments. A TODO comment that *becomes outdated* 1 describes a feature that is not being implemented, which means that such a comment should be addressed within some deadline, and then removed from the code base (S14) when either the respective code is written and potentially documented with a different comment, or the feature is abandoned altogether. At the same time, more research nowadays is conducted to understand the relations between different QAs.

Mapping taxonomies. In recent years, several taxonomies concerning code comments have been proposed, however, all of them are characterized by a rather different focus, such as the scope of the comments (S02), the information embedded in the comment (S29, S41), the issues related to specific comment types (S06, S33, S40), as well as the programming language they belong to. This suggests the need for a comprehensive code comment taxonomy or model that maps all these aspects and definitions in a more coherent manner to have a better overview of developer commenting practices across languages. Rani *et al.* adapted the code comment taxonomies of Java and Python (S29, S41) for class comments of Java and Python [76]. They mapped the taxonomies to Smalltalk class comments and found that developers write similar kinds of information in class comments across languages. Such a mapping can encourage building language-independent approaches for other aspects of comment quality evaluation.

5. Implication for Future studies 3

Besides the aspects discussed above, future studies on code 4 comment assessment should be devoted to filling the gaps of the last decade of research as well as coping with the needs of developers interested in leveraging comment assessment tools in different program languages.

Investigating specific comment types (RQ1). Several works 5 showed the importance of different types of comments to achieve specific development tasks and understanding about code. Although, the trend of analyzing specific comment types

has increased over the last decade, there are still comment types 6 (*e.g.*, class and package comments) that need more attention.

Generalizing across languages (RQ1). Given the preponderance of studies focusing on the Java language, and considering that statistics from various developer boards (StackOverflow, GitHub) suggest that there are other popular languages as well (*e.g.*, Python and JavaScript), more studies on analyzing various types of comments in these languages are needed. 7 Interesting questions in this direction could concern the comparison of practices (*e.g.*, given Python is often considered to be “self-explainable”, do developers write fewer comments in Python?) and tools used to write code comments in different languages (*e.g.*, popularity of Javadoc v.s. Pydoc). Similarly, whether various programming language paradigms, such as functional versus object-oriented languages, or statically-typed versus dynamic-typed languages, play a role in the way developers embed information in comments, or the way they treat comments, needs further work in this direction.

Identifying QAs (RQ2). Our results show various QAs, *e.g.*, 8 *consistency*, *completeness*, and *accuracy* that are frequently considered in assessing comment quality. Additionally, various metrics, tools, and techniques that are proposed to assess them automatically. Indeed, some QAs are largely overlooked in the literature, *e.g.*, there is not enough research on approaches and automated tools that ensure that comments are *accessible*, *trustworthy*, and *understandable*, despite numerous studies suggesting that having good code comments brings several benefits.

Standardizing QAs (RQ2). We identify various QAs that researchers consider assessing comment quality. Not all of these QAs are unique *i.e.*, they have conceptual overlap (based on their definitions in Table 4 and measurement techniques in Table 6). For example, the definition of *up-to-datedness* and *consistency* mention of keeping comments updated. Similarly, the definition of *coherence* and *similarity* focus on the relatedness between code and comments. In this study, we mainly focus on identifying various QAs from the literature and on extracting

metrics, tools, and techniques to measure them. Standardizing their definition can be an essential next step in the direction of comment quality assessment research. Since not every study provides the definition of mentioned QAs, such a work will require surveying the authors to understand how they perceive various QAs and where they refer to for QAs definitions.

Comment smells (RQ2). Although there is no standard definition of good or bad comments, many studies indicate bloated comments (or non-informative comments), redundant comments (contain same information as in the code), or inconsistent comments (*e.g.*, contain conflicting information compared to the code) as code or comment smells. Arnaoudva *et al.* identified various LAs that developers perceive as poor practices and should be avoided [59]. Still, what information is vital in comments is a subjective concept and can sometimes be contradictory. For instance, Oracle’s coding style guideline suggests including author information in class comments, whereas the Apache style guideline suggests removing it as it can be inferred from the version control system [87]. We find that researchers use the *completeness* QA to identify informative comments. They define various metrics to assess the completeness of comments, as shown in Table 7. These metrics check the presence of specific information, such as summary, author, or exception information in class or method comments. Future work can investigate the definition of good and bad comments by surveying various sources, such as documentation guidelines, researchers, and developers, and comparing the sources across to improve the understanding of high-quality comments. Such work can inspire the development of more metrics and tools to ensure the adherence of comments to the standards.

Automated tools and techniques (RQ3). Finally, concerning techniques to assess comment quality, we observed that those based on AI, such as NLP and ML, were increasingly used in the past decade. On the other hand, deep learning techniques do not yet seem to have gained a foothold within the community for assessing comment quality. Since code comment generation is becoming more and more popular also due to such advanced

techniques emerging, we envision that future work may study techniques and metrics to assess the quality of automatically generated code comments.

Research evaluation (RQ4 and RQ5). Scientific methods play a crucial role in the growth of engineering knowledge [88]. Several studies have indicated the weak validation in software engineering [22]. We also find that several studies propose solutions but do not evaluate their solution. Also, various approaches were validated only by the authors of the work or by surveying students. However, we need to do all steps as engineering science researchers do, empirically investigating the problems, proposing solutions, and validating those solutions.

In contrast to seven research types listed in Table 5, we observe only limited types of research studies. For example, we do not find any philosophical, opinion, or experience papers for the comment quality assessment domain even though this domain is more than a decade old now. Philosophical papers sketch a new perspective of looking at things, conceptual frameworks, metrics *etc.* Opinion papers present good or bad opinions of authors about something, such as different approaches to assess quality, using particular frameworks *etc.* Similarly, experience papers often present insights about lessons learned or anecdotes by authors in using tools or techniques in practice. Such papers help tool designers better shape their future tools.

6. Threats to validity

We now outline potential threats to the validity of our study. *Threats to construct validity* mainly concern the measurements used in the evaluation process. In this case, threats can be mainly due to (i) the imprecision in the automated selection of relevant papers (*i.e.*, the three-step search on the conference proceedings based on regular expressions), and to (ii) the subjectivity and error-proneness of the subsequent manual classification and categorization of relevant papers.

We mitigated the first threat by manually classifying a sample of relevant papers from a set of conference proceedings and compared this classification with the one recommended by the

automated approach based on regular expressions. This allowed us to incrementally improve the initial set of regular expressions. To avoid any bias in the selection of the papers, we selected regular expression in a deterministic way (as detailed in the section 2): We first examined the definition of *documentation* and *comment* in *IEEE Standard Glossary of Software Engineering Terminology* (IEEE Standard 610.12-1990) and identified the first set of keywords *comment*, *documentation*, and *specification*; we further added comment-related keywords that are frequently mentioned in the context of code comments. Moreover, we formulated a set of keywords to discard irrelevant studies that presented similar keywords (e.g., code review comments). To verify the correctness of the final set of keywords, we manually scanned the full venue proceedings metadata to make sure the set of keywords did not prune relevant papers. This iterative approach allowed us to verify that our keyword-based filtering approach does not lead to false negatives for the selected venues.

We mitigated the second threat by applying multi-stage manual classification of conference proceedings, involving multiple evaluators and reviewers, as detailed in section 2.

Threats to internal validity concern confounding factors that could influence our results and findings. A possible source of bias might be related to the way we selected and analyzed the conference proceedings. To deal with potential threats regarding the actual regular expressions considered for the selection of relevant studies, we created regular expressions that tend to be very inclusive, *i.e.*, that select papers that are marginally related to the topic of interest, and we take a final decision only after a manual assessment.

Threats to external validity concern the generalization and completeness of results and findings. Although the number of analyzed papers is large, since it involves studies spanning the last ten years of research, there is still the possibility that we missed some relevant studies. We mitigate this threat by applying various selection criteria to select relevant conference proceedings, considering the well-established venues and communities related to code comment-related studies, as detailed

in section 2. It is important to mention that this paper intentionally limits its scope in two ways, which threatens to the completeness of the study results and findings. First of all, we mainly focus on research work investigating code comment quality without integrating studies from industry tracks of conference venues (as was done in previous studies thematically close to ours [17, 18]). Second, we focus on those studies that involve manually written code comments in order to avoid auto-generated comments (already investigated in recent related work [89, 90]). To further limit potential threats concerning the completeness of our study, we use the snowball approach to reach potentially relevant studies that we could have missed with our venue selection. However, we support the argument of Garousi et al. [91] who report that a *multivocal* literature review, with further replications, is desirable to make the overall interpretation of code comment quality attributes more complete for future work.

7. Related Work

This section discusses the literature concerning (i) studies motivating the importance of quality attributes for software documentation, (ii) comment quality aspects, and (iii) recent SLRs discussing topics closely related to our investigation.

Important quality attributes for software documentation.

Various research works conducted surveys with developers to identify important quality attributes of good software documentation. Forward and Lethbridge surveyed 48 developers, and highlighted developer concerns about outdated documentation [92]. Chen and Huang surveyed 137 project managers and software engineers [93]. Their study highlighted the typical quality problems developers face in maintaining software documentation: adequacy, complete, traceability, consistency, and trustworthiness. Robillard *et al.* conducted personal interviews with 80 practitioners and presented the important attributes for good documentation, such as including examples and usage information, complete, organized, and better design [94]. Similarly, Plosch *et al.* surveyed 88 practitioners and identified

consistency, clarity, accuracy, readability, organization, and understandability as the most important attributes [95]. They also indicated that developers do not consider documentation standards important (e.g., ISO 26514:2008, IEEE Std.1063:2001). Sohan *et al.* in their survey study highlighted the importance of examples in documentation [96]. The majority of the highlighted documentation quality attributes apply to code comments as well (as a type of software documentation). However, which specific quality attributes (e.g., outdated, complete, consistent, traceable) researchers consider important to assess code comment quality and how these quality attributes are measured is yet to study.

Comment quality. Evaluating comment quality according to various aspects has gained a lot of attention from researchers, for instance, assessing their adequacy [97] and their content quality [10, 11], analyzing co-evolution of comments and code [98], or detecting inconsistent comments [12, 14]. Several works have proposed tools and techniques for the automatic assessment of comment quality [10, 11, 99]. For instance, Khamis *et al.* assessed the quality of inline comments based on consistency and language quality using a heuristic-based approach [10]. Steidl *et al.* evaluated documentation comment quality based on four quality attributes, such as consistency, coherence, completeness, and usefulness of comments using a machine learning-based model [11]. Zhou *et al.* proposed a heuristic and natural language processing-based technique to detect incomplete and incorrect comments [16]. These works have proposed various new quality attributes to assess comment quality, such as completeness, coherence, and language quality, that are not included in previous quality models. However, a unifying overview of comment QAs and their assessment approaches is still missing. Our paper complements these previous works by investigating comment QAs discussed in the last decade of research.

Previous SLRs on code comments and software documentation. In recent years, SLRs have been conducted to investigate agile software development aspects in open-source projects [100], the usage of ontologies in software process as-

essment [101], and improvement aspects in DevOps process and practices [102]. Previous SLRs in the field investigated code comments and software documentation [17, 18], which are closely related to our work. Specifically, Ding *et al.* conducted an SLR to explore the usage of knowledge-based approaches in software documentation [17]. They identified twelve QAs. They also highlighted the need to improve QAs, especially conciseness, credibility, and unambiguity. Zhi *et al.* have explored various types of software documentation to see which QAs impact it [18]. Both of the studies considered the timeline until 2011. Additionally, they have not studied how the proposed comment quality assessment approaches are computed in practice for comments. Inspired by these related studies, we focused specifically on the code comment aspect. Song *et al.* conducted a literature review on code comment generation techniques, and indicated the need to design an objective comment quality assessment model [89]. Complementarily, Nazar *et al.* [90] presented a literature review in the field of summarizing software artifacts, which included source code comment generation as well as bug reports, mailing lists, and developer discussion artifacts. Our work complements these previous studies since we mainly focus on manually written comments.

8. Conclusion

In this work, we present the results of a systematic literature review on source code comment quality evaluation practices in the decade 2011—2020. We study 47 publications to understand of effort of Software Engineering researchers, in terms of what type of comments they focus their studies on, what QAs they consider relevant, what techniques they resort to in order to assess their QAs, and finally, how they evaluate their contributions. Our findings show that most studies consider only comments in Java source files, and thus may not generalize to comments of other languages, and they focus on only a few QAs, especially on consistency between code and comments. Some QAs, such as conciseness, coherence, organization, and usefulness, are rarely investigated. As coherent and concise comments play an important role in program understanding, es-

establishing approaches to assess these attributes requires more attention from the community. We also observe that the majority of the approaches appear to be based on heuristics rather than machine learning or other techniques and, in general, need better evaluation. Such approaches require validation on other languages and projects to generalize them. Though the trend of analyzing comments appearing in multiple projects and languages is increasing compared to the previous decade, as reported by Zhi *et al.*, the approaches still need more thorough validation[18].

Acknowledgement 2

We gratefully acknowledge the financial support of the Swiss National Science Foundation for the project “Agile Software Assistance” (SNSF project No. 200020-181973, Feb 1, 2019 - Apr 30, 2022) and the Spanish Government through the SCUM grant RTI2018-102043-B-I00, and the Madrid Regional through the project BLOQUES. We also acknowledge the Horizon 2020 (EU Commission) support for the project *COSMOS* (DevOps for Complex Cyber-physical Systems), Project No. 957254-COSMOS.

References 4

- [1] M. Abidi, F. Khomh, [Towards the definition of patterns and code smells for multi-language systems](#), in: EuroPLoP '20: European Conference on Pattern Languages of Programs 2020, Virtual Event, Germany, 1-4 July, 2020, ACM, 2020, pp. 37:1–37:13. doi:10.1145/3424771.3424792. URL <https://doi.org/10.1145/3424771.3424792>
- [2] M. Lehman, D. Perry, J. Ramil, W. Turski, P. Wernick, Metrics and laws of software evolution—the nineties view, in: Proceedings IEEE International Software Metrics Symposium (METRICS'97), IEEE Computer Society Press, Los Alamitos CA, 1997, pp. 20–32. doi:10.1109/METRIC.1997.637156.
- [3] M. Törnigren, U. Sellgren, [Complexity Challenges in Development of Cyber-Physical Systems](#), Springer International Publishing, Cham, 2018, pp. 478–503. doi:10.1007/978-3-319-95246-8_27. URL https://doi.org/10.1007/978-3-319-95246-8_27
- [4] S. C. B. de Souza, N. Anquetil, K. M. de Oliveira, A study of the documentation essential to software maintenance, in: Proceedings of the 23rd annual international conference on Design of communication: documenting & designing for pervasive information, SIGDOC '05, ACM,

- New York, NY, USA, 2005, pp. 68–75. doi:10.1145/1085313.1085331.
- [5] U. Dekel, J. D. Herbsleb, Reading the documentation of invoked API functions in program comprehension, in: 2009 IEEE 17th International Conference on Program Comprehension, IEEE, 2009, pp. 168–177.
- [6] C. McMillan, D. Poshyanyk, M. Grechanik, Recommending source code examples via API call usages and documentation, in: Proceedings of the 2nd International Workshop on Recommendation Systems for Software Engineering, 2010, pp. 21–25.
- [7] L. Tan, D. Yuan, G. Krishna, Y. Zhou, /* iComment: Bugs or bad comments?*/, in: Proceedings of twenty-first ACM SIGOPS symposium on Operating systems principles, 2007, pp. 145–158.
- [8] M. Allamanis, E. T. Barr, C. Bird, C. Sutton, [Learning natural coding conventions](#), in: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014, ACM, New York, NY, USA, 2014, pp. 281–293. doi:10.1145/2635868.2635883. URL <http://doi.acm.org/10.1145/2635868.2635883>
- [9] B. W. Kernighan, R. Pike, [The Practice of Programming \(Addison-Wesley Professional Computing Series\)](#), 1st Edition, Addison-Wesley, 1999. URL <http://www.amazon.com/exec/obidos/redirect?tag=citeulike07-20&path=ASIN/020161586X>
- [10] N. Khamis, R. Witte, J. Rilling, Automatic quality assessment of source code comments: the JavadocMiner, in: International Conference on Application of Natural Language to Information Systems, Springer, 2010, pp. 68–79.
- [11] D. Steidl, B. Hummel, E. Juergens, Quality analysis of source code comments, in: Program Comprehension (ICPC), 2013 IEEE 21st International Conference on, IEEE, 2013, pp. 83–92.
- [12] I. K. Ratol, M. P. Robillard, Detecting fragile comments, in: Proceedings of the 32Nd IEEE/ACM International Conference on Automated Software Engineering, IEEE Press, 2017, pp. 112–122.
- [13] L. Pascarella, A. Bacchelli, [Classifying code comments in Java open-source software systems](#), in: Proceedings of the 14th International Conference on Mining Software Repositories, MSR '17, IEEE Press, 2017, pp. 227–237. doi:10.1109/MSR.2017.63. URL <https://doi.org/10.1109/MSR.2017.63>
- [14] F. Wen, C. Nagy, G. Bavota, M. Lanza, A large-scale empirical study on code-comment inconsistencies, in: Proceedings of the 27th International Conference on Program Comprehension, IEEE Press, 2019, pp. 53–64.
- [15] Y. Padioleau, L. Tan, Y. Zhou, Listening to programmers — taxonomies and characteristics of comments in operating system code, in: Proceedings of the 31st International Conference on Software Engineering, IEEE Computer Society, 2009, pp. 331–341.
- [16] Y. Zhou, R. Gu, T. Chen, Z. Huang, S. Panichella, H. Gall, Analyzing APIs documentation and code to detect directive defects, in: Proceedings of the 39th International Conference on Software Engineering,

- IEEE Press, 2017, pp. 27–37. 1
- [17] W. Ding, P. Liang, A. Tang, H. Van Vliet, Knowledge-based approaches in software documentation: A systematic literature review, *Information and Software Technology* 56 (6) (2014) 545–567. 2
- [18] J. Zhi, V. Garousi-Yusifoglu, B. Sun, G. Garousi, S. Shahnewaz, G. Ruhe, Cost, benefits and quality of software development documentation: A systematic mapping, *Journal of Systems and Software* 99 (2015) 175–198. 3
- [19] B. Kitchenham, S. Charters, Guidelines for performing systematic literature reviews in software engineering (2007). 4
- [20] S. Keele, Guidelines for performing systematic literature reviews in software engineering, Tech. rep., Technical report, EBSE Technical Report EBSE-2007-01 (2007). 5
- [21] S. Y. Auyang, Engineering-an endless frontier, Harvard University Press, 2006. 6
- [22] M. V. Zelkowitz, D. Wallace, Experimental validation in software engineering, *Information and Software Technology* 39 (11) (1997) 735–743. 7
- [23] M. Petticrew, H. Roberts, Systematic reviews in the social sciences: A practical guide, John Wiley & Sons, 2008. 8
- [24] S. Stapleton, Y. Gambhir, A. LeClair, Z. Eberhart, W. Weimer, K. Leach, Y. Huang, A human study of comprehension and code summarization, in: ICPC '20: 28th International Conference on Program Comprehension, Seoul, Republic of Korea, July 13–15, 2020, ACM, 2020, pp. 2–13. doi:10.1145/3387904.3389258. URL <https://doi.org/10.1145/3387904.3389258> 9
- [25] M. Kuhrmann, D. M. Fernández, M. Daneva, On the pragmatic design of literature studies in software engineering: An experience-based guideline, *Empirical Softw. Engg.* 22 (6) (2017) 2852–2891. doi:10.1007/s10664-016-9492-y. URL <https://doi.org/10.1007/s10664-016-9492-y> 10
- [26] H. Hata, C. Treude, R. G. Kula, T. Ishio, 9.6 million links in source code comments: Purpose, evolution, and decay, in: Proceedings of the 41st International Conference on Software Engineering, IEEE Press, 2019, pp. 1211–1221. 11
- [27] E. Aghajani, C. Nagy, M. Linares-Vásquez, L. Moreno, G. Bavota, M. Lanza, D. C. Shepherd, Software documentation: the practitioners' perspective, in: 2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE), IEEE, 2020, pp. 590–601. 12
- [28] D. Haouari, H. A. Sahraoui, P. Langlais, How good is your comment? A study of comments in Java programs, in: Proceedings of the 5th International Symposium on Empirical Software Engineering and Measurement, ESEM 2011, Banff, AB, Canada, September 22–23, 2011, IEEE Computer Society, 2011, pp. 137–146. doi:10.1109/ESEM.2011.22. URL <https://doi.org/10.1109/ESEM.2011.22> 13
- [29] G. Garousi, V. Garousi, M. Moussavi, G. Ruhe, B. Smith, Evaluating usage and quality of technical software documentation: An empirical study, in: F. Q. B. da Silva, N. J. Juzgado, G. H. Travassos (Eds.), 17th International Conference on Evaluation and Assessment in Software Engineering, EASE '13, Porto de Galinhas, Brazil, April 14–16, 2013, ACM, 2013, pp. 24–35. doi:10.1145/2460999.2461003. URL <https://doi.org/10.1145/2460999.2461003> 15
- [30] R. Pandita, X. Xiao, H. Zhong, T. Xie, S. Oney, A. M. Paradkar, Inferring method specifications from natural language API descriptions, in: M. Glinz, G. C. Murphy, M. Pezzè (Eds.), 34th International Conference on Software Engineering, ICSE 2012, June 2–9, 2012, Zurich, Switzerland, IEEE Computer Society, 2012, pp. 815–825. doi:10.1109/ICSE.2012.6227137. URL <https://doi.org/10.1109/ICSE.2012.6227137> 16
- [31] B. Dagenais, M. P. Robillard, Using traceability links to recommend adaptive changes for documentation evolution, *IEEE Trans. Software Eng.* 40 (11) (2014) 1126–1146. doi:10.1109/TSE.2014.2347969. URL <https://doi.org/10.1109/TSE.2014.2347969> 17
- [32] D. Fucci, A. Mollaalizadehbahnemiri, W. Maalej, On using machine learning to identify knowledge in API reference documentation, in: Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 109–119. 18
- [33] S. Scalabrino, G. Bavota, C. Vendome, M. L. Vásquez, D. Poshvanyk, R. Oliveto, Automatically assessing code understandability: how far are we?, in: G. Rosu, M. D. Penta, T. N. Nguyen (Eds.), Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering, ASE 2017, Urbana, IL, USA, October 30 - November 03, 2017, IEEE Computer Society, 2017, pp. 417–427. doi:10.1109/ASE.2017.8115654. URL <https://doi.org/10.1109/ASE.2017.8115654> 19
- [34] S. Fakhoury, Y. Ma, V. Arnaudova, O. O. Adesope, The effect of poor source code lexicon and readability on developers' cognitive load, in: F. Khomh, C. K. Roy, J. Siegmund (Eds.), Proceedings of the 26th Conference on Program Comprehension, ICPC 2018, Gothenburg, Sweden, May 27–28, 2018, ACM, 2018, pp. 286–296. doi:10.1145/3196321.3196347. URL <https://doi.org/10.1145/3196321.3196347> 20
- [35] E. Aghajani, C. Nagy, G. Bavota, M. Lanza, A large-scale empirical study on linguistic antipatterns affecting APIs, in: 2018 IEEE International Conference on Software Maintenance and Evolution, ICSME 2018, Madrid, Spain, September 23–29, 2018, IEEE Computer Society, 2018, pp. 25–35. doi:10.1109/ICSME.2018.00012. URL <https://doi.org/10.1109/ICSME.2018.00012> 21
- [36] H. Li, S. Li, J. Sun, Z. Xing, X. Peng, M. Liu, X. Zhao, Improving API caveats accessibility by mining API caveats knowledge graph, in: 2018 IEEE International Conference on Software Maintenance and Evolution, ICSME 2018, Madrid, Spain, September 23–29, 2018, IEEE Computer Society, 2018, pp. 183–193. doi:10.1109/ICSME.2018.00028. URL <https://doi.org/10.1109/ICSME.2018.00028> 22
- [37] C. Wang, X. Peng, M. Liu, Z. Xing, X. Bai, B. Xie, T. Wang, A learning-based approach for automatic construction of domain glossary 23

- from source code and documentation, in: M. Dumas, D. Pfahl, S. Apel, A. Russo (Eds.), Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019, ACM, 2019, pp. 97–108. doi:10.1145/3338906.3338963. URL <https://doi.org/10.1145/3338906.3338963>
- [38] P. Nie, R. Rai, J. J. Li, S. Khurshid, R. J. Mooney, M. Gligoric, A framework for writing trigger-action todo comments in executable format, in: M. Dumas, D. Pfahl, S. Apel, A. Russo (Eds.), Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/SIGSOFT FSE 2019, Tallinn, Estonia, August 26-30, 2019, ACM, 2019, pp. 385–396. doi:10.1145/3338906.3338965. URL <https://doi.org/10.1145/3338906.3338965>
- [39] E. Aghajani, C. Nagy, O. L. Vega-Márquez, M. Linares-Vásquez, L. Moreno, G. Bavota, M. Lanza, Software documentation issues unveiled, in: J. M. Atlee, T. Bultan, J. Whittle (Eds.), Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, Montreal, QC, Canada, May 25-31, 2019, IEEE / ACM, 2019, pp. 1199–1210. doi:10.1109/ICSE.2019.00122. URL <https://doi.org/10.1109/ICSE.2019.00122>
- [40] T. M. T. Pham, J. Yang, The secret life of commented-out source code, in: ICPC ’20: 28th International Conference on Program Comprehension, Seoul, Republic of Korea, July 13-15, 2020, ACM, 2020, pp. 308–318. doi:10.1145/3387904.3389259. URL <https://doi.org/10.1145/3387904.3389259>
- [41] X. Sun, Q. Geng, D. Lo, Y. Duan, X. Liu, B. Li, Code comment quality analysis and improvement recommendation: an automated approach, International Journal of Software Engineering and Knowledge Engineering 26 (06) (2016) 981–1000.
- [42] J. Zhai, X. Xu, Y. Shi, G. Tao, M. Pan, S. Ma, L. Xu, W. Zhang, L. Tan, X. Zhang, CPC: automatically classifying and propagating natural language comments via program analysis, in: G. Rothermel, D. Bae (Eds.), ICSE ’20: 42nd International Conference on Software Engineering, Seoul, South Korea, 27 June - 19 July, 2020, ACM, 2020, pp. 1359–1371. doi:10.1145/3377811.3380427. URL <https://doi.org/10.1145/3377811.3380427>
- [43] M. M. Rahman, C. K. Roy, I. Keivanloo, Recommending insightful comments for source code using crowdsourced knowledge, in: M. W. Godfrey, D. Lo, F. Khomh (Eds.), 15th IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2015, Bremen, Germany, September 27-28, 2015, IEEE Computer Society, 2015, pp. 81–90. doi:10.1109/SCAM.2015.7335404. URL <https://doi.org/10.1109/SCAM.2015.7335404>
- [44] S. Scalabrino, M. Linares-Vasquez, D. Poshyanyk, R. Oliveto, Improving code readability models with textual features, in: 2016 IEEE 24th International Conference on Program Comprehension (ICPC), IEEE, 2016, pp. 1–10.
- [45] P. W. McBurney, C. McMillan, Automatic source code summarization of context for Java methods, IEEE Trans. Software Eng. 42 (2) (2016) 103–119. doi:10.1109/TSE.2015.2465386. URL <https://doi.org/10.1109/TSE.2015.2465386>
- [46] Y. Zhou, C. Wang, X. Yan, T. Chen, S. Panichella, H. C. Gall, Automatic detection and repair recommendation of directive defects in Java API documentation, IEEE Trans. Software Eng. 46 (9) (2020) 1004–1023. doi:10.1109/TSE.2018.2872971. URL <https://doi.org/10.1109/TSE.2018.2872971>
- [47] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, S. Li, Measuring program comprehension: A large-scale field study with professionals, IEEE Trans. Software Eng. 44 (10) (2018) 951–976. doi:10.1109/TSE.2017.2734091. URL <https://doi.org/10.1109/TSE.2017.2734091>
- [48] G. Garousi, V. Garousi-Yusifoglu, G. Ruhe, J. Zhi, M. Moussavi, B. Smith, Usage and usefulness of technical software documentation: An industrial case study, Information and Software Technology 57 (2015) 664–682.
- [49] M. Monperrus, M. Eichberg, E. Tekes, M. Mezini, What should developers be aware of? An empirical study on the directives of API documentation, Empir. Softw. Eng. 17 (6) (2012) 703–737. doi:10.1007/s10664-011-9186-4. URL <https://doi.org/10.1007/s10664-011-9186-4>
- [50] Y. Wu, Y. Manabe, T. Kanda, D. M. Germán, K. Inoue, Analysis of license inconsistency in large collections of open source projects, Empir. Softw. Eng. 22 (3) (2017) 1194–1222. doi:10.1007/s10664-016-9487-8. URL <https://doi.org/10.1007/s10664-016-9487-8>
- [51] L. Pascarella, M. Bruntink, A. Bacchelli, Classifying code comments in Java software systems, Empir. Softw. Eng. 24 (3) (2019) 1499–1537. doi:10.1007/s10664-019-09694-w. URL <https://doi.org/10.1007/s10664-019-09694-w>
- [52] Y. Zhou, X. Yan, W. Yang, T. Chen, Z. Huang, Augmenting Java method comments generation with context information based on neural networks, J. Syst. Softw. 156 (2019) 328–340. doi:10.1016/j.jss.2019.07.087. URL <https://doi.org/10.1016/j.jss.2019.07.087>
- [53] A. D. Lucia, M. D. Penta, R. Oliveto, Improving source code lexicon via traceability and information retrieval, IEEE Trans. Software Eng. 37 (2) (2011) 205–227. doi:10.1109/TSE.2010.89. URL <https://doi.org/10.1109/TSE.2010.89>
- [54] H. Zhong, Z. Su, Detecting API documentation errors, in: A. L. Hosking, P. T. Eugster, C. V. Lopes (Eds.), Proceedings of the 2013 ACM SIGPLAN International Conference on Object Oriented Programming Systems Languages & Applications, OOPSLA 2013, part of SPLASH 2013, Indianapolis, IN, USA, October 26-31, 2013, ACM, 2013, pp. 803–816. doi:10.1145/2509136.2509523.

- URL <https://doi.org/10.1145/2509136.2509523> 1
- [55] Y. Shinyama, Y. Arahori, K. Gondow, Analyzing code comments to boost program comprehension, in: 2018 25th Asia-Pacific Software Engineering Conference (APSEC), IEEE, 2018, pp. 325–334. 2
- [56] M. P. Robillard, Y. B. Chhetri, [Recommending reference API documentation](#), *Empir. Softw. Eng.* 20 (6) (2015) 1558–1586. doi:10.1007/s10664-014-9323-y. 3
URL <https://doi.org/10.1007/s10664-014-9323-y>
- [57] G. M. Rama, A. C. Kak, [Some structural measures of API usability](#), *Softw. Pract. Exp.* 45 (1) (2015) 75–110. doi:10.1002/spe.2215. 4
URL <https://doi.org/10.1002/spe.2215>
- [58] P. W. McBurney, C. McMillan, [An empirical study of the textual similarity between source code and source code summaries](#), *Empir. Softw. Eng.* 21 (1) (2016) 17–42. doi:10.1007/s10664-014-9344-6. 5
URL <https://doi.org/10.1007/s10664-014-9344-6>
- [59] V. Arnaudova, M. D. Penta, G. Antoniol, [Linguistic antipatterns: what they are and how developers perceive them](#), *Empir. Softw. Eng.* 21 (1) (2016) 104–158. doi:10.1007/s10664-014-9350-8. 6
URL <https://doi.org/10.1007/s10664-014-9350-8>
- [60] A. Corazza, V. Maggio, G. Scanniello, Coherence of comments and method implementations: A dataset and an empirical investigation, *Software Quality Journal* 26 (2) (2018) 751–777. 7
- [61] S. Scalabrino, M. Linares-Vásquez, R. Oliveto, D. Shybyanyk, A comprehensive model for code readability, *Journal of Software: Evolution and Process* 30 (6) (2018) e1958. 8
- [62] Z. Liu, H. Chen, X. Chen, X. Luo, F. Zhou, [Automatic detection of outdated comments during code changes](#), in: S. Reisman, S. I. Ahmed, C. Demartini, T. M. Conte, L. Liu, W. R. Claycomb, M. Nakamura, E. Tovar, S. Cimato, C. Lung, H. Takakura, J. Yang, T. Akiyama, Z. Zhang, K. Hasan (Eds.), 2018 IEEE 42nd Annual Computer Software and Applications Conference, COMPSAC 2018, Tokyo, Japan, 23-27 July 2018, Volume 1, IEEE Computer Society, 2018, pp. 154–163. doi:10.1109/COMPSAC.2018.00028. 9
URL <https://doi.org/10.1109/COMPSAC.2018.00028>
- [63] J. Zhang, L. Xu, Y. Li, [Classifying python code comments based on supervised learning](#), in: X. Meng, R. Li, K. Wang, B. Niu, X. Wang, G. Zhao (Eds.), Web Information Systems and Applications - 15th International Conference, WISA 2018, Taiyuan, China, September 14-15, 2018, Proceedings, Vol. 11242 of Lecture Notes in Computer Science, Springer, 2018, pp. 39–47. doi:10.1007/978-3-030-02934-0_4. 10
URL https://doi.org/10.1007/978-3-030-02934-0_4
- [64] L. Pascarella, A. Ram, A. Nadeem, D. Bisesser, N. Knyazev, A. Bacchelli, [Investigating type declaration mismatches in Python](#), in: F. A. Fontana, B. Walter, A. Ampatzoglou, F. Palomba (Eds.), 2018 IEEE Workshop on Machine Learning Techniques for Software Quality Evaluation, MaLTeSQuE, SANER 2018, Campobasso, Italy, March 20, 2018, IEEE Computer Society, 2018, pp. 43–48. doi:10.1109/MALTESQUE.2018.8368458. 11
- URL <https://doi.org/10.1109/MALTESQUE.2018.8368458> 12
- [65] M. Kechagia, M. Fragkoulis, P. Louridas, D. Spinellis, [The exception handling riddle: An empirical study on the Android API](#), *J. Syst. Softw.* 142 (2018) 248–270. doi:10.1016/j.jss.2018.04.034. 13
URL <https://doi.org/10.1016/j.jss.2018.04.034>
- [66] Y. Xi, L. Shen, Y. Gui, W. Zhao, Migrating deprecated API to documented replacement: Patterns and tool, in: Proceedings of the 11th Asia-Pacific Symposium on Internetware, 2019, pp. 1–10. 14
- [67] M. Iammarino, L. Aversano, M. L. Bernardi, M. Cimitile, [A topic modeling approach to evaluate the comments consistency to source code](#), in: 2020 International Joint Conference on Neural Networks, IJCNN 2020, Glasgow, United Kingdom, July 19-24, 2020, IEEE, 2020, pp. 1–8. doi:10.1109/IJCNN48605.2020.9207651. 15
URL <https://doi.org/10.1109/IJCNN48605.2020.9207651>
- [68] O. A. L. Lemos, M. Suzuki, A. C. de Paula, C. L. Goes, [Comparing identifiers and comments in engineered and non-engineered code: A large-scale empirical study](#), in: C. Hung, T. Cerný, D. Shin, A. Bechini (Eds.), SAC '20: The 35th ACM/SIGAPP Symposium on Applied Computing, online event, [Brno, Czech Republic], March 30 - April 3, 2020, ACM, 2020, pp. 100–109. doi:10.1145/3341105.3373972. 16
URL <https://doi.org/10.1145/3341105.3373972>
- [69] R. Wieringa, N. Maiden, N. Mead, C. Rolland, Requirements engineering paper classification and evaluation criteria: a proposal and a discussion, *Requirements engineering* 11 (1) (2006) 102–107. 17
- [70] K. Petersen, R. Feldt, S. Mujtaba, M. Mattsson, Systematic mapping studies in software engineering, in: 12th International Conference on Evaluation and Assessment in Software Engineering (EASE) 12, 2008, pp. 1–10. 18
- [71] J. M. González-Barahona, G. Robles, [On the reproducibility of empirical software engineering studies based on data retrieved from development repositories](#), *Empirical Software Engineering* 17 (1) (2012) 75–89. doi:10.1007/s10664-011-9181-9. 19
URL <http://dx.doi.org/10.1007/s10664-011-9181-9>
- [72] M. Motwani, Y. Brun, [Automatically generating precise oracles from structured natural language specifications](#), in: J. M. Atlee, T. Bultan, J. Whittle (Eds.), Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, IEEE / ACM, 2019, pp. 188–199. doi:10.1109/ICSE.2019.00035. 20
URL <https://doi.org/10.1109/ICSE.2019.00035>
- [73] R. S. Malik, J. Patra, M. Pradel, [NI2type: inferring JavaScript function types from natural language information](#), in: J. M. Atlee, T. Bultan, J. Whittle (Eds.), Proceedings of the 41st International Conference on Software Engineering, ICSE 2019, IEEE / ACM, 2019, pp. 304–315. doi:10.1109/ICSE.2019.00045. 21
URL <https://doi.org/10.1109/ICSE.2019.00045>
- [74] E. Nurvitadhi, W. W. Leung, C. Cook, Do class comments aid Java program understanding?, in: 33rd Annual Frontiers in Education, 2003. FIE 2003., Vol. 1, IEEE, 2003, pp. T3C–T3C. 22

- [75] P. Rani, S. Panichella, M. Leuenberger, M. Ghafari, O. Nierstrasz, *What do class comments tell us? An investigation of comment evolution and practices in Pharo Smalltalk*, Empirical Software Engineering 26 (6) (2021) 1–49. [arXiv:2005.11583](#), [doi:10.1007/s10664-021-09981-5](#).
URL <http://scg.unibe.ch/archive/papers/Rani21b.pdf>
- [76] P. Rani, S. Panichella, M. Leuenberger, A. Di Sorbo, O. Nierstrasz, *How to identify class comment types? A multi-language approach for class comment classification*, Journal of Systems and Software 181 (2021) 111047. [arXiv:2107.04521](#), [doi:https://doi.org/10.1016/j.jss.2021.111047](#).
URL <http://scg.unibe.ch/archive/papers/Rani21d.pdf>
- [77] F. Tomassetti, M. Torchiano, An empirical assessment of polyglot-ism in GitHub, in: Proceedings of the 18th International Conference on Evaluation and Assessment in Software Engineering, 2014, pp. 1–4.
- [78] M. Visconti, C. R. Cook, Assessing the state of software documentation practices, in: International Conference on Product Focused Software Process Improvement, Springer, 2004, pp. 485–496.
- [79] S. W. Ambler, Agile/lean documentation: strategies for agile software development, Retrieved June 20 (2007) 2007.
- [80] A. Dautovic, R. Plösch, M. Saft, Automated quality defect detection in software development documents, in: First International Workshop on Model-Driven Software Migration (MDSM 2011), 2011, p. 29.
- [81] W. Maalej, R. Tiarks, T. Roehm, R. Koschke, *On the comprehension of program comprehension*, ACM TOSEM 23 (4) (2014) 31:1–31:37. [doi:10.1145/2622669](#).
URL <http://mobis.informatik.uni-hamburg.de/wp-content/uploads/2014/06/TOSEM-Maalej-Comprehension-PrePrint2.pdf>
- [82] R. Kallis, A. Di Sorbo, G. Canfora, S. Panichella, Predicting issue types on GitHub, Science of Computer Programming 205 (2021) 102598.
- [83] S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu, J. Gao, Deep learning-based text classification: A comprehensive review, ACM Computing Surveys (CSUR) 54 (3) (2021) 1–40.
- [84] S. E. S. Committee, et al., Ieee standard for a software quality metrics methodology, IEEE Std 1061-1992 (1993) 1–96 [doi:10.1109/IEEESTD.1993.115124](#).
- [85] N. Fenton, J. Bieman, Software metrics: a rigorous and practical approach, CRC press, 2014.
- [86] A. Meneely, B. H. Smith, L. A. Williams, *Validating software metrics: A spectrum of philosophies*, ACM Trans. Softw. Eng. Methodol. 21 (4) (2012) 24:1–24:28. [doi:10.1145/2377656.2377661](#).
URL <https://doi.org/10.1145/2377656.2377661>
- [87] *Oracle documentation guidelines*, <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html> (verified on 10 Sep 2020).
URL <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>
- [88] W. G. Vincenti, et al., What engineers know and how they know it, Vol. 141, Baltimore: Johns Hopkins University Press, 1990.
- [89] X. Song, H. Sun, X. Wang, J. Yan, A survey of automatic generation of source code comments: Algorithms and techniques, IEEE Access 7 (2019) 111411–111428.
- [90] N. Nazar, Y. Hu, H. Jiang, Summarizing software artifacts: A literature review, Journal of Computer Science and Technology 31 (5) (2016) 883–909.
- [91] V. Garousi, M. Felderer, M. V. Mäntylä, *The need for multivocal literature reviews in software engineering: complementing systematic literature reviews with grey literature*, in: S. Beecham, B. A. Kitchenham, S. G. MacDonell (Eds.), Proceedings of the 20th International Conference on Evaluation and Assessment in Software Engineering, EASE 2016, Limerick, Ireland, June 01 - 03, 2016, ACM, 2016, pp. 26:1–26:6. [doi:10.1145/2915970.2916008](#).
URL <https://doi.org/10.1145/2915970.2916008>
- [92] A. Forward, T. C. Lethbridge, The relevance of software documentation, tools and technologies: A survey, in: Proceedings of the 2002 ACM symposium on Document engineering, DocEng '02, ACM, New York, NY, USA, 2002, pp. 26–33. [doi:10.1145/585058.585065](#).
- [93] J.-C. Chen, S.-J. Huang, An empirical analysis of the impact of software development problem factors on software maintainability, Journal of Systems and Software 82 (6) (2009) 981–992.
- [94] M. P. Robillard, *What makes APIs hard to learn? answers from developers*, IEEE Softw. 26 (6) (2009) 27–34. [doi:10.1109/MS.2009.193](#).
URL <https://doi.org/10.1109/MS.2009.193>
- [95] R. Plösch, A. Dautovic, M. Saft, *The value of software documentation quality*, in: 2014 14th International Conference on Quality Software, Allen, TX, USA, October 2-3, 2014, IEEE, 2014, pp. 333–342. [doi:10.1109/QSIC.2014.22](#).
URL <https://doi.org/10.1109/QSIC.2014.22>
- [96] S. Sohan, F. Maurer, C. Anslow, M. P. Robillard, A study of the effectiveness of usage examples in REST API documentation, in: 2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC), IEEE, 2017, pp. 53–61.
- [97] J. D. Arthur, K. T. Stevens, Assessing the adequacy of documentation through document quality indicators, in: Proceedings. Conference on Software Maintenance-1989, IEEE, 1989, pp. 40–49.
- [98] B. Fluri, M. Würsch, E. Giger, H. C. Gall, Analyzing the co-evolution of comments and source code, Software Quality Journal 17 (4) (2009) 367–394.
- [99] H. Yu, B. Li, P. Wang, D. Jia, Y. Wang, Source code comments quality assessment method based on aggregation of classification algorithms, J. Comput. Appl. 36 (12) (2016) 3448–3453.
- [100] A. Silva, T. Araújo, J. Nunes, M. Perkusich, E. Dilorenzo, H. O. de Almeida, A. Perkusich, *A systematic review on the use of definition of done on agile software development projects*, in: E. Mendes, S. Counsell, K. Petersen (Eds.), Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering, EASE

2017, Karlskrona, Sweden, June 15-16, 2017, ACM, 2017, pp. 364–373. doi:10.1145/3084226.3084262.
URL <https://doi.org/10.1145/3084226.3084262>

[101] A. Tarhan, G. Giray, On the use of ontologies in software process assessment: A systematic literature review, in: E. Mendes, S. Counsell, K. Petersen (Eds.), Proceedings of the 21st International Conference on Evaluation and Assessment in Software Engineering, EASE 2017, Karlskrona, Sweden, June 15-16, 2017, ACM, 2017, pp. 2–11. doi:10.1145/3084226.3084261.
URL <https://doi.org/10.1145/3084226.3084261>

[102] S. Badshah, A. A. Khan, B. Khan, Towards process improvement in devops: A systematic literature review, in: J. Li, L. Jaccheri, T. Dingsoyr, R. Chitchyan (Eds.), EASE '20: Evaluation and Assessment in Software Engineering, Trondheim, Norway, April 15-17, 2020, ACM, 2020, pp. 427–433. doi:10.1145/3383219.3383280.
URL <https://doi.org/10.1145/3383219.3383280>